Communications Issues

Inefficient or incorrect communication strategies can overwhelm a system. For example, some process migration implementations employ systemwide broadcasts. The flood of broadcast messages can overwhelm communication channels. Due to communication delays, many overloaded processors could receive a request for a process at the same time and all send their processes to one underloaded processor.[188]

Several strategies have been devised to avoid these problems. For example, the algorithm could restrict processors to communicate only with then immediate neighbors, which reduces the number of messages that are transmitted, but increases the time required for information to reach every node in the system.[189] Alternatively, processors could periodically select a random processor with which to exchange information. In this case, processes are migrated from the processor with the higher load to the one with a lower load.[190] In cases where one processor is severely overloaded and the rest are underloaded, this results in rapid process diffusion.

Figure 15.17 illustrates process diffusion in a system in which nodes communicate only with their neighbors. The overloaded processor, represented by the middle vertex, has 17 processes while all others have one process. After one iteration, the overloaded processor communicates with its neighbors and sends three processes to each. Now those processors have four processes, which is three more than some of their neighbors have. In the second iteration, the processors with four processes send some to their neighbors. Finally, in the third iteration, the overloaded processor once again sends some processes to its neighbors so that now the processor with the heaviest load has only three processes. This example illustrates that even when communications are kept among neighboring processors, load balancing can effectively distribute processing responsibilities throughout the system.

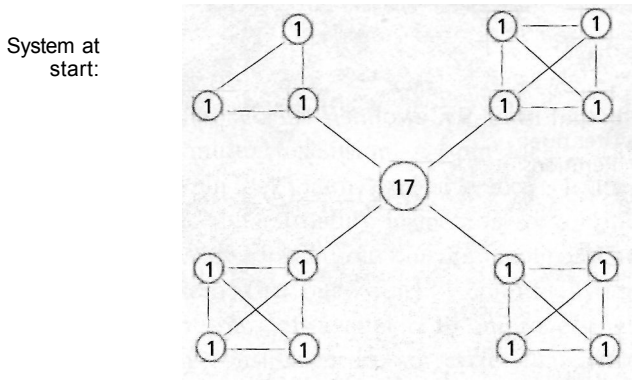
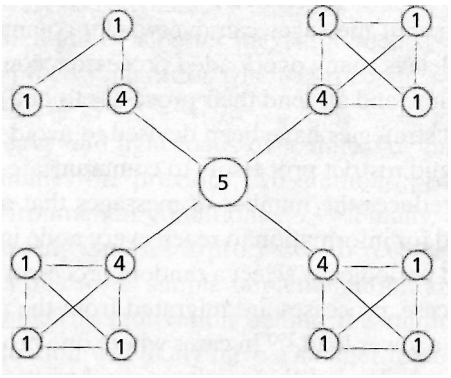
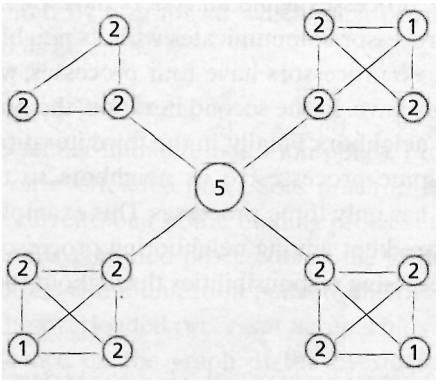


Figure 15.17 | *Processor load diffusion. (Part 1 of 2.)*

... After one iteration



...After two iterations



...After three iterations:

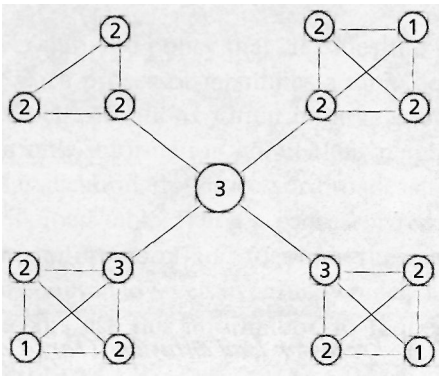


Figure 15.17 *| Processor load diffusion. (Part 2 of 2.)*

Self Review

1. In what type of environment should a sender-oriented policy be used? A receiver-oriented policy? Justify your answers.
2. How does high communication latency hinder the effectiveness of load balancing?

Ans: **1)** A system with a light load should use a sender-oriented policy, whereas a system with a heavy load should use a receiver-oriented policy. In both cases this will reduce unnecessary migrations, because there will be few senders in a system with a light load and few receivers in one with a heavy load. **2)** A node sending a message that it is underloaded might have become overloaded by the time another node receives the message. Thus, the receiving node could further unbalance the system's load by migrating additional processes to the sender.

15.9 Multiprocessor Mutual Exclusion

Many of the mutual exclusion mechanisms we described in Chapter 5 are not adequate for multiprocessor systems. For example, disabling interrupts, which prevents other processes from executing by disabling preemption on uniprocessor systems, does not guarantee mutual exclusion on multiprocessors because several processes can execute simultaneously on different processors. Instructions such as test-and-set can be inefficient when the data these instructions reference is located in remote memory. Test-and-set has another weakness, namely that each of the pending operations requires a separate memory access; these are performed sequentially, because only one access to a memory location may normally occur at a time. Such collisions can quickly saturate various kinds of interconnection networks, causing serious performance problems. Because of nontrivial communication costs and the fact that multiple processes execute at the same time, designers have developed a number of mutual exclusion techniques for multiprocessors. This section introduces mutual exclusion techniques for nondistributed multiprocessing systems. We present distributed mutual exclusion in Section 17.5, Mutual Exclusion in Distributed Systems.

15.9.1 Spin Locks

Multiprocessor operating systems, such as Windows XP and Linux, frequently use **spin locks** for multiprocessor mutual exclusion. A spin lock is called a "lock" because a process that holds a spin lock claims exclusive access to the resource the spin lock protects (e.g., a shared data structure in memory or a critical section of code). In effect, other processes are "locked out" of the resource. Other processes that wish to use the resource "spin" (i.e., busy wait) by continually testing a condition to determine whether the resource is available. In uniprocessing systems, spin locks are wasteful because they consume processor cycles that could be used by other processes, including the one that holds the lock, to do useful work. In multiprocessing systems, the process holding the lock could release it while another process is busy waiting. In this case, it is more efficient for a process to busy wait if the time required for the system to perform a context switch to schedule another process is longer than the average busy-wait time.[191] Also, if a processor contains no

other runnable processes in its run queue, it makes sense to keep the process spinning to minimize the response time when the spin lock is released.[192,193]

When a process holds a spin lock for a long duration (relative to context-switch time), blocking is more efficient. **Delayed blocking** is a technique in which a process spins for a short period of time; if it does not acquire the lock in that time, the process blocks.[194] An **advisable process lock (APL)** presents an alternative solution. When a process acquires an APL, it specifies the amount of time it will hold the lock. Based on the time specified, other processes waiting to acquire the APL can determine if it is more efficient to busy wait for the lock or to block.[195]

Adaptive locks (also called **configurable locks)** add flexibility to a mutual exclusion implementation. At certain times, such as when there are few active processes, spin locks are preferable because a process can acquire a resource soon after it becomes available. At times when system load is high, spinning wastes valuable processor cycles and blocking is preferable. Adaptive locks permit a process to dynamically change the type of lock being used. These locks can be set to blocking, spinning or delayed blocking and can incorporate APL features to customize the lock according to system load and application needs.[196]

When multiple processes simultaneously wait for a spin lock to be released, indefinite postponement can occur. Operating systems can prevent indefinite postponement by granting the spin lock to processes in first-come-first-served order, or by aging processes waiting on the spin lock, as discussed in Section 7.3.

Self Review

1. In what way can spinning be useful in a multiprocessor? Under what conditions is it not desirable in a multiprocessor? Why is spinning not useful in a uniprocessor system?
2. Why might an APL not be useful in all situations?

Ans: **1)** Spinning can be useful because it minimizes the time between when a resource becomes available and when a new process acquires the resource. Spinning is not desirable when system load is high and the expected time before the resource is available is greater than context-switch time; in these cases, spinning uses valuable processor cycles. Spinning is not useful in uniprocessor systems because it blocks the execution of the process holding the resource. **2)** An APL is not useful when a process cannot determine how long it will hold the lock.

15.9.2 Sleep/Wakeup Locks

A sleep/wakeup lock provides synchronization similar to a spin lock, but reduces wasted processor cycles and bus traffic. Consider processes P_1 and P_2, both of which request use of a resource protected by a sleep/wakeup lock. P_1 requests the resource first and obtains the lock. When P_2 requests the resource owned by P_1 and does not receive the resource, P_2 responds by sleeping (i.e., blocking). Once P_1 releases the lock, P_1 wakes the highest-priority process waiting for the resource (in this case, P_2). Unlike spin locks (which give the lock to the next waiting process), sleep/wakeup locks can use the processor scheduler to enforce process priorities.

Sleep/wakeup locks can cause waiting processes to be indefinitely postponed, depending on the system's scheduling policy.

Note that unlike a uniprocessor implementation, only the process with the highest priority is awakened. When all threads are awakened, a **race condition** can result because two or more threads may access a resource associated with a lock in a nondeterministic order. Race conditions should be avoided because they can cause subtle errors in applications and are difficult to debug. In uniprocessors, even if all processes are alerted, there can be no race condition because only one process (the one with highest priority in most scheduling algorithms) obtains control of the processor and acquires the lock. In a multiprocessing environment, a broadcast could wake many processes that are competing for the lock, creating a race condition. This would also result in one process obtaining the lock and many processes testing, reblocking and consequently going back to sleep, thus squandering processor time due to context switching. This phenomenon is known as the **thundering herd.**[197]

Self Review

1. What are an advantage and a disadvantage of a sleep/wakeup lock compared to a spin lock?
2. How is the implementation of a sleep/wakeup lock different in a multiprocessor environment than in a uniprocessor environment?

Ans: **1)** A sleep/wakeup lock eliminates wasted processor cycles incurred by spinning. Also, a sleep/wakeup lock ensures that the highest-priority waiting process obtains the lock next. A disadvantage is that reaction to the lock's availability is slower because the new acquirer must awaken and obtain a processor. **2)** In a multiprocessor environment, a releasing process awakens only one waiting process, which prevents a thundering herd that would waste processor cycles and flood communication channels. This is not a problem on uniprocessor systems.

15.9.3 Read/Write Locks

Enforcing mutually exclusive access to shared memory in a multiprocessor system can degrade performance. Only writers need exclusive access to the resource, whereas, in general, multiple readers can access the same memory location at once. Therefore, many systems protect shared memory with a more versatile **read/write** lock rather than a generic mutual exclusion lock.[198] A read/write lock provides mutual exclusion similar to that presented in the readers/writers problem of Section 6.2.4. A read/write lock permits multiple reader processes (i.e., processes that will not alter shared data) to enter their critical sections. Unlike the example in Section 6.2.4, however, read/write locks require that a writer (i.e., a process that will alter shared data) wait until there are no reader or writer processes in their critical sections before entering its critical section.[199]

To implement this locking approach efficiently, the mutual exclusion mechanism must use shared memory. However, in environments in which memory is not shared, such as NORMA systems, message passing must be used. We discuss such techniques in Section 17.5, Mutual Exclusion in Distributed Systems.

Self Review

1. In what situations are read/write locks more efficient than spin locks?
2. A naive implementation of a read/write lock allows any reader to enter its critical section when no writer is in its critical section. How can this lead to indefinite postponement for writers? What is a fairer implementation?

Ans: **1)** Read/write locks are more efficient than spin locks when multiple processes read a memory location, but do not write to it. In this case, multiple processes can be in critical sections at once. **2)** This implementation leads to indefinite postponement if readers continually enter their critical sections before a writer can enter its critical section. A fairer implementation would allow a reader to enter its critical section only if no writer is in or is waiting to enter its critical section as discussed in Section 6.2.4.

Web Resources

`www.winntmag.com/Articles/Index.cfm?ArticleID=303&pg=l&show=495`
This article describes multiprocessor scheduling in Windows NT.

`www.aceshardware.com/Spades/read.php?article_id=30000187`
This article describes how the AMD Athlon (with two processors) implements cache coherence. It also discusses bus snooping.

www.mosix.org
This site provides information about MOSIX, a software bundle used on UNIX systems, which implements automatic process migration and load balancing.

www.teamten.com/lawrence/242.paper/242.paper.html
This paper discusses mutual exclusion algorithms for multiprocessors.

`users.win.be/W0005997/UNIX/LINUX/IL/atomicity-eng.html`
This site explains how to use spin locks to implement mutual exclusion in Linux.

Summary

Many applications demand substantially more computing power than one processor can provide. As a result, multiprocessing systems—computers that use more than one processor to meet a system's processing needs—are often employed. The term "multiprocessing system" encompasses any system with more than one processor. This includes dual-processor personal computers, powerful servers that contain many processors and distributed groups of workstations that work together to perform tasks.

There are several ways to classify multiprocessors. Flynn developed an early scheme for classifying computers into increasingly parallel configurations based on the types of streams used by processors. SISD (single instruction stream, single data stream) computers are traditional uniprocessors that fetch one instruction at a time and execute it on a single data item. MISD (multiple instruction stream, single data stream) computers (which are not commonly used) have multiple processing units, each operating on a piece of data and then passing the result to the next processing unit. SIMD (single instruction stream, multiple data stream) computers, which include array and vector processors, execute instructions on multiple data items in parallel. MIMD (multiple instruction stream, multiple data stream) computers have multiple processing units that operate independently on separate instruction streams.

The interconnection scheme of a multiprocessor system describes how the system physically connects its components, such as processors and memory modules. The interconnection scheme affects the system's performance, reliability and cost, so it is a key issue for multiprocessor designers. A shared bus provides low cost and high performance for a small number of processors but does not scale well, owing to contention on the bus as the number of processors increases. A crossbar-switch matrix provides high fault tolerance and performance but is inappropriate for small systems in which UMA (uniform memory access) is more cost effective. A 2-D mesh network is a simple design that provides adequate performance and fault tolerance at

a low cost but that also does not scale well. A hypercube is more scalable than a 2-D mesh network and provides better performance, but at a higher cost. Multistage networks compromise between cost and performance and can be used to build extremely large-scale multiprocessors.

In a tightly coupled system, the processors share most system resources. Loosely coupled systems connect components indirectly through communication links and do not share most resources. Loosely coupled systems are more scalable, flexible and fault tolerant but do not perform as well as tightly coupled systems. Loosely coupled systems also place more burden on programmers, who must implement applications that communicate via IPC rather than shared memory.

Multiprocessors can also be categorized based on how the processors share operating system responsibilities. In the master/slave organization, only the master processor can execute the operating system; the slaves execute only user programs. In the separate-kernels organization, each processor executes its own operating system, and operating system data structures maintain the system's global information. In the symmetrical organization, all processors can control any device and reference any storage unit. This organization enables systems to balance their workloads more precisely.

Multiprocessor systems use several architectures to share memory. UMA multiprocessors require all processors to share all the system's main memory equally owing to shared-memory contention; these systems are not scalable beyond a few processors. NUMA (nonuniform memory access) multiprocessors partition memory into modules, assigning one to each processor as its local memory. COMA (cache-only memory architecture) multiprocessors are similar to NUMA multiprocessors, but treat all memory as a large cache to increase the likelihood that requested data resides in the requesting processor's local memory. NORMA (no remote memory access) multiprocessors are loosely coupled and do not provide any globally shared main memory. NORMA multiprocessors are used to build large-scale distributed systems.

Memory is coherent if the value obtained from reading a memory address is always the same as the value most recently written to that address. UMA cache-coherence protocols include bus snooping and directory-based coherence. CC-NUMAs (cache-coherent NUMAs) often use a home-based approach in which a home node for each memory address is responsible for keeping that data item coherent throughout the system.

Systems that use page replication maintain multiple copies of a memory page so it can be accessed quickly by multiple processors. Systems that use page migration transfer pages to the processor that accesses the pages most. These techniques can be combined to optimize performance.

Shared virtual memory (SVM) provides the illusion of shared physical memory in large-scale multiprocessor systems. Invalidation (in which a writer voids other copies of a page) and write broadcast (in which a writer notifies other processors of updates to a page) are two approaches for implementing memory coherence in SVM systems. These protocols can be strictly applied, but this can be inefficient. Relaxed consistency, in which the system might not be coherent for a few seconds, improves efficiency but sacrifices some data integrity.

Multiprocessor scheduling algorithms must determine both when and where to dispatch a process. Some algorithms maximize processor affinity (the relationship of a process to a particular processor and its local memory) by executing related processes on the same processors. Others maximize parallelism by executing related processes on separate processors simultaneously. Some systems use global run queues, whereas others (typically large-scale systems) maintain per-processor or per-node run queues.

Process migration is the act of transferring a process between two processors or computers. Process migration can increase performance, resource sharing and fault tolerance. To migrate a process, nodes must transfer the process's state, which includes the process's memory pages, register contents, state of open files and kernel context. Migration policies that allow residual dependency (a process's dependency on its former node) decrease fault tolerance and the process's performance after migration. However, eliminating residual dependency makes initial migration slow. Various migration policies balance the goals of minimal residual dependency and fast initial migration.

Load balancing is a technique in which the system attempts to distribute processing responsibility equally among processors to increase processor utilization and decrease process response times. Static load balancing algorithms assign a fixed number of processors to a job when the job is first scheduled. These algorithms are useful in environments, such as scientific computing, where process interactions and execution times are predictable. Dynamic load balancing algorithms change the number of processors assigned to a job throughout the job's life. These algorithms are useful when process interactions are unpredictable, and when processes can be created and terminate at any time.

Many uniprocessor mutual exclusion mechanisms are either inefficient or ineffective for multiprocessors. Design-

ers have developed mutual exclusion techniques specific to multiprocessors. A spin lock is a mutual exclusion lock in which waiting processes spin (i.e., busy wait) for the lock. This is appropriate when system load is low and spinning time is short relative to context-switch time. Spin locks reduce reaction time when resources become available. With a sleep/wakeup lock, a process releasing the lock wakes up the highest-priority waiting process, which acquires the lock. This reduces the use of processor cycles typical in spin locks. Read/write locks allow one writer or many readers to be in their critical sections at once.

Key Terms

2-D mesh network—Multiprocessor interconnection scheme that arranges nodes in an *m* x *n* rectangle.

4-connected 2-D mesh network—2-D mesh network in which nodes are connected with the nodes directly to the north, south, east and west.

acquire operation—In several coherence strategies, an operation indicating that a process is about to access shared memory.

adaptive lock—Mutual exclusion lock that allows processes to switch between using a spin lock or a blocking lock, depending on the current condition of the system.

advisable process lock (APL)—Locking mechanism in which an acquirer estimates how long it will hold the lock; other processes can use this estimate to determine whether to block or spin when waiting for the lock.

array processor—SIMD (single instruction stream, multiple data stream) system consisting of many (possibly tens of thousands) simple processing units, each executing the same instruction in parallel on many data elements.

attraction memory (AM)—Main memory in a COMA (cache-only memory architecture) multiprocessor, which is organized as a cache.

baseline network—Type of multistage network.

bidding algorithm—Dynamic load balancing algorithm in which processors with smaller loads "bid" for jobs on overloaded processors; the bid value depends on the load of the bidding processor and the distance between the underloaded and overloaded processors.

bisection width—Minimum number of links that need to be severed to divide a network into two unconnected halves.

bus snooping—Coherence protocol in which processors "snoop" the shared bus to determine whether a requested write is to a data item in that processor's cache or, if applicable, local memory.

cache coherence—Property of a system in which any data item read from a cache has the value equal to the last write to that data item.

cache-coherent NUMA (CC-NUMA)—NUMA multiprocessor that maintains cache coherence, usually through a home-based approach.

cache miss latency—Extra time required to access data that does not reside in the cache.

cache-only memory architecture (COMA) multiprocessor - Multiprocessor architecture in which nodes consist of a processor, cache and memory module; main memory is organized as a large cache.

cache snooping—Bus snooping used to ensure cache coherency.

configurable lock—See adaptive lock

contention—In multiprocessing a situation in which several processors compete for the use of a shared resource.

copy-on-reference migration—Process migration technique in which only a process's dirty pages are migrated with the process, and the process can request clean pages either from the sending node or from secondary storage.

cost of an interconnection scheme—Total number of links in a network.

coscheduling—Job-aware process scheduling algorithm that attempts to execute processes from the same job concurrently by placing them in adjacent global-run-queue locations.

crossbar-switch matrix—Processor interconnection scheme that maintains a separate path from every sender node to every receiver node.

degree of a node—Number of other nodes with which a node is directly connected.

delayed blocking—Technique whereby a process spins on a lock for a fixed amount of time before it blocks; the rationale is that if the process does not obtain the lock quickly, it will probably have to wait a long time, so it should block.

delayed consistency—Memory coherence strategy in which processors send update information after a release, but a

receiving node does not apply this information until it performs an acquire operation on the memory.

dirty eager migration—Process migration method in which only a process's dirty pages are migrated with the process; clean pages must be accessed from secondary storage.

drafting algorithm—Dynamic load balancing algorithm that classifies each processor's load as low, normal or high; each processor maintains a table of the other processors' loads, and the system uses a receiver-initiated policy to exchange processes.

dynamic load balancing—Technique that attempts to distribute processing responsibility equally by changing the number of processors assigned to a job throughout the job's life.

dynamic partitioning—Job-aware process scheduling algorithm that divides processors in the system evenly among jobs, except that no single job can be allocated more processors than runnable processes; this algorithm maximizes processor affinity.

eager migration—Process migration strategy that transfers the entire address space of a process during the initial phases of migration to eliminate a migrated process's residual dependencies on its original node.

false sharing—Situation that occurs when processes on separate processors are forced to share a page because they are each accessing a data item on that page, although not the same data item.

first-come-first-served (FCFS) process scheduling—Job-blind multiprocessor scheduling algorithm that places arriving processes in a queue; the process at the head of the queue executes until it freely relinquishes the processor.

flushing—Process migration strategy in which the sending node writes all of the process's memory pages to a shared disk at the start of migration; the process then accesses the pages from the shared disk as needed on the receiving node.

gang scheduling—Another name for coscheduling.

global run queue—Process scheduling queue used in some multiprocessor scheduling algorithms, which is independent of the processors in a system and into which every process or job in the system is placed.

hard affinity—Type of processor affinity in which the scheduling algorithm guarantees that a process only executes on a single node throughout its life cycle.

home-based consistency—Memory-coherence strategy in which processors send coherence information to a home node associated with the page being written; the home node forwards update information to other nodes that subsequently access the page.

home node—Node that is the "home" for a physical memory address or page and is responsible for maintaining the data's coherence.

hypercube—Multiprocessor interconnection scheme that consists of 2^n nodes (where n is an integer); each node is linked with n neighbor nodes.

interconnection scheme—Design that describes how a multiprocessor system physically connects its components, such as processors and memory modules.

invalidation—Memory-coherence protocol in which a process first invalidates—i.e., voids—all other copies of a page before writing to the page.

job-aware scheduling—Multiprocessor scheduling algorithms that account for job properties when making scheduling decisions; these algorithms typically attempt to maximize parallelism or processor affinity.

job-blind scheduling—Multiprocessor scheduling algorithms that ignore job properties when making scheduling decisions; these algorithms are typically simple to implement.

lazy copying—Process migration strategy that transfers pages from the sender only when the process at the receiving node references these pages.

lazy data propagation—Technique in which writing processors send coherence information after a release, but not the data; a processor retrieves the data when it accesses a page that it knows is not coherent.

lazy migration—Process migration strategy in multiprocessor systems that does not transfer all pages during initial migration. This increases residual dependency but reduces initial migration time.

lazy release consistency—Memory-coherence strategy in which a processor does not send coherence information after writing to a page until a new processor attempts an acquire operation on that memory page.

load balancing—Technique which attempts to distribute the system's processing responsibility equally among processors.

loosely coupled system—System in which processors do not share most resources; these systems are flexible and fault tolerant but perform worse than tightly coupled systems.

massively parallel processor—Processor that performs a large number of instructions on large data sets at once; array processors are often called massively parallel processors.

master/slave multiprocessor organization—Scheme for delegating operating system responsibilities in which only one

processor (the "master") can execute the operating system, and the other processors (the "slaves") can execute only user processes.

memory coherence—State of a system in which the value obtained from reading a memory address is always the same as the most-recently written value at that address.

memory line—Entry in memory that stores one machine word of data, which is typically four or eight bytes.

multiple-instruction-stream, multiple-data-stream (MIMD) computer—Computer architecture consisting of multiple processing units, which execute independent instructions and manipulate independent data streams; this design describes multiprocessors.

multiple-instruction-stream, single-data-stream (MISD) computer—Computer architecture consisting of several processing units, which execute independent instruction streams on a single stream of data; these architectures have no commercial application.

multiple shared bus architecture—Interconnection scheme that employs several shared buses connecting processors and memory. This reduces contention, but increases cost compared to a single shared bus.

multiprocessing system—Computing system that employs more than one processor.

multistage network—Multiprocessor interconnection scheme that uses switch nodes as hubs for communication between processor nodes that each have their own local memory.

network diameter—Shortest path between the two most remote nodes in a system.

network link—Connection between two nodes.

node—System component, such as a processor, memory module or switch, attached to a network; sometimes a group of components might be viewed as a single node.

nonuniform-memory-access (NUMA) multiprocessor—Multiprocessor architecture in which each node consists of a processor, cache and memory module. Access to a processor's associated memory module (called local memory) is faster than access to other memory modules in the system.

no-remote-memory-access (NORMA) multiprocessor—Multiprocessor architecture that does not provide global shared memory. Each processor maintains its own local memory. NORMA multiprocessors implement a common shared virtual memory.

on-demand migration—Another name for lazy process migration.

page migration—Technique in which the system transfers pages to the processor (or processors when used with page replication) that accesses the pages most.

page replication—Technique in which the system maintains multiple copies of a page at different nodes so that it can be accessed quickly by multiple processors.

per-processor run queue—Process scheduling queue associated with a specific processor; processes entering the queue are scheduled on the associated processor independently of the scheduling decisions made in rest of the system.

per-node run queue—Process scheduling queue associated with a group of processors; processes entering the queue are scheduled on the associated node's processors independently of the scheduling decisions made in rest of the system.

precopy—Process migration strategy in which the sender begins transferring dirty pages before the original process is suspended; once the number of untransferred dirty pages at the sender reaches some threshold, the process migrates.

process migration—Transferring a process and its associated state between two processors.

processor affinity—Relationship of a process to a particular processor and a corresponding memory bank.

race condition—Occurs when multiple threads simultaneously compete for the same serially reusable resource, and that resource is allocated to these threads in an indeterminate order. This can cause subtle program errors when the order in which threads access a resource is important.

random policy—Dynamic load balancing policy in which the system arbitrarily chooses a processor to receive a migrated process.

read/write lock—Lock that allows a single writer process or multiple reading processes (i.e., processes that will not alter shared variables) to enter a critical section.

receiver-initiated policy—Dynamic load balancing policy in which processors with low utilization attempt to find overloaded processors from which to receive a migrated process.

relaxed consistency—Category of memory-coherence strategies that permit the system to be in an incoherent state for a few seconds after a write, but improve performance over strict consistency.

release consistency—Memory-coherence strategy in which multiple accesses to shared memory are considered a single access; these accesses begin with an acquire and end

with a release, after which coherence is enforced throughout the system.

release operation—In several coherence strategies, this operation indicates that a process is done accessing shared memory.

residual dependency—Dependency of a migrated process on its original node after process migration because some of the process's state remains on the original node.

round-robin job (RRJob) scheduling—Job-aware process scheduling algorithm employing a global run queue in which jobs are dispatched to processors in a round-robin fashion.

round-robin process (RRprocess) scheduling—Job-blind multiprocessor scheduling algorithm that places each process in a global run queue and schedules these process in a round-robin manner.

sender-initiated policy—Dynamic load balancing policy in which overloaded processors attempt to find underloaded processors to which to migrate a process.

separate kernels multiprocessor organization—Scheme for delegating operating system responsibilities in which each processor executes its own operating system, but the processors share some global system information.

sequential consistency—Category of memory-coherence strategies in which coherence protocols are enforced immediately after a write to a shared memory location.

shared bus—Multiprocessor interconnection scheme that uses a single communication path to connect all processors and memory modules.

shared virtual memory (SVM)—An extension of virtual memory concepts to multiprocessor systems; SVM presents the illusion of shared physical memory between processors and ensures coherence for pages accessed by separate processors.

shortest-process-first (SPF) scheduling (multiprocessor)—Job-blind multiprocessor scheduling algorithm, employing a global run queue, that selects the process with the smallest processor time requirement to execute on an available processor.

single-instruction-stream, multiple-data-stream (SIMD) computer—Computer architecture consisting of one or more processing elements that execute instructions from a single instruction stream that act on multiple data items.

single-instruction-stream, single-data-stream (SISD) computer—Computer architecture in which one processor fetches instructions from a single instruction stream and manipulates a single data stream; this architecture describes traditional uniprocessors.

sleep/wakeup lock—Mutual exclusion lock in which waiting processes block, and a releasing process wakes the highest-priority waiting process and gives it the lock.

smallest-number-of-processes-first (SNPF) scheduling—Job-aware process scheduling algorithm, employing a global job-priority queue, where job priority is inversely proportional to the number of processes in a job.

soft affinity—Type of processor affinity in which the scheduling algorithm tries, but does not guarantee, to schedule a process only on a single node throughout its life cycle.

space-partitioning scheduling—Multiprocessor scheduling strategy that attempts to maximize processor affinity by scheduling collaborative processes on a single processor (or single set of processors); the underlying assumption is that these processes will access the same shared data.

spin lock—Mutual exclusion lock in which a waiting process busy waits for the lock; this reduces the process's reaction time when the protected resource's becomes available.

static load balancing—Category of load balancing algorithms that assign a fixed number of processors to a job when it is first scheduled.

stream—Sequence of objects fed to the processor.

switch—Node that routes messages between component nodes.

symmetric multiprocessor (SMP)—Multiprocessor system in which processors share all resources equally, including memory, I/O devices and processes.

symmetrical multiprocessor organization—Scheme for delegating operating system responsibilities in which each processor can execute the single operating system.

symmetric policy—Dynamic load balancing policy that combines the sender-initiated policy and the receiver-initiated policy to provide maximum versatility to adapt to environmental conditions.

thundering herd—Phenomenon that occurs when many processes awaken when a resource becomes available; only one process acquires the resource, and the others test the lock's availability and reblock, wasting processor cycles.

tightly coupled system—System in which processors share most resources; these systems provide higher performance but are less fault tolerant and flexible than loosely coupled systems.

timesharing scheduling—Multiprocessor scheduling technique that attempts to maximize parallelism by scheduling collaborative processes concurrently on different processors.

undivided coscheduling algorithm—Job-aware process scheduling algorithm in which processes of the same job are placed in adjacent locations in the global run queue, and processes are scheduled round-robin.

uniform-memory-access (UMA) multiprocessor—Multiprocessor architecture that requires all processors to share all of main memory; in general, memory-access time is constant, regardless of which processor requests data, except when the data is stored in a processor's cache.

vector processor—Type of SIMD computer containing one processing unit that executes instructions that operate on multiple data items.

write broadcast—Technique for maintaining memory coherence in which the processor that performs a write broadcasts the write throughout the system.

Exercises

15.1 A multiprocessor's interconnection scheme affects the system's performance, cost and reliability.

a. Of the schemes presented in this chapter, which are best suited for small systems and which for large-scale systems?

b. Why are some interconnection schemes good for small networks and others for large networks, but none are optimal for all networks?

15.2 For each of the following multiprocessor organizations, describe an environment in which the organization is useful; also, list a drawback for each.

a. Master/slave multiprocessor organization

b. Separate-kernels multiprocessor organization

c. Symmetrical organization

15.3 For each of the following environments, suggest whether the UMA, NUMA or NORMA memory-access architecture would be best and explain why.

a. An environment consisting of a few interactive processes that communicate using shared memory

b. Thousands of workstations performing a common task

c. Large-scale multiprocessor containing 64 processors in a single machine

d. Dual-processor personal computer

15.4 As we described in the chapter, one important, but difficult, goal of a NUMA multiprocessor designer is maximizing the number of page accesses that can be serviced by local memory. Describe the three strategies, COMA, page migration and page replication, and discuss the advantages and disadvantages of each.

a. COMA multiprocessor.

b. Page migration.

c. Page replication.

15.5 For each of the following multiprocessor scheduling algorithms, use the classifications discussed early in this chapter to describe the type of multiprocessor system that would likely employ it. Justify your answers.

a. Job-blind multiprocessor scheduling

b. Job-aware multiprocessor scheduling

c. Scheduling using per-processor or per-node run queues

15.6 For each of the following system attributes, describe how process migration can increase it in a system and an inefficient implementation reduce it.

a. Performance

b. Fault tolerance

c. Scalability

15.7 In our load balancing discussion, we described how processors decide when and with whom to migrate a process. However, we did not describe how processors decide which process to migrate. Suggest some factors that could help determine which process to migrate. *[Hint:* Consider the benefits of process migration other than load balancing.]

15.8 Can a process waiting for a spin lock be indefinitely postponed even if all processes guarantee to leave their critical sections after a finite amount of time? A process waiting for a sleep/wakeup lock? If you answered yes, suggest a way to prevent indefinite postponement.

Suggested Projects

15.9 Prepare a research paper describing how the Linux operating system supports CC-NUMA multiprocessors. Describe Linux's scheduling algorithm, how Linux maintains cache and memory coherency and the various mutual exclusion mechanisms provided by the operating system. Make sure your research entails reading some source code.

15.10 The cost and performance of different hardware devices are changing at different rates. For example, hardware continues to get cheaper, and processor speed is increasing faster than bus speed. Write a research paper describing the trends in interconnection schemes. Which schemes are becoming more popular? Which less popular? Why?

15.11 Prepare a research paper surveying the load balancing algorithms in use today. What are the motivations behind each algorithm? For what type of environment is each intended? Are most algorithms static or dynamic?

15.12 Prepare a research paper describing how operating systems implement memory coherence. Include a precise description of the coherence protocol and when it is applied for at least one real operating system.

Suggested Simulations

15.13 Use Java threads to simulate a multiprocessor, with each thread representing one processor. Be sure to synchronize access to global (i.e., shared) variables. Implement shared memory as an array with memory addresses as subscripts. Define an object class called Process. Randomly create Process objects and implement a scheduling algorithm for them. A Process object should describe how long it runs before blocking each time, and when it terminates. It should also specify when it requires access to a shared memory location and to which location.

15.14 Expand your simulation. Implement per-processor run queues and a load balancing algorithm. Maintain a data structure for a processor's local memory, and implement a function in the Process object to randomly add memory locations to the local memory. Be sure to migrate this local memory when migrating a Process.

Recommended Reading

Flynn and Rudd provide a synopsis of parallel and sequential architecture classifications in their 1996 paper "Parallel Architectures."[200] Crawley, in his paper "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," surveys processor interconnection schemes.[201]

As multiprocessing systems have become more mainstream, much research has been devoted to optimizing their performance. Mukherjee et al. present a survey of multiprocessor operating system concepts and systems in "A Survey of Multiprocessor Operating Systems."[202] More recently Tripathi and Karnik, in their paper "Trends in Multiprocessor and Distributed Operating System Designs," summarize many important topics such as scheduling, memory access and locks.[203]

For more information regarding scheduling algorithms in multiprogrammed environments, see Leutenegger and Vernon's "The Performance of Multiprogrammed Multiprocessor Scheduling Policies,"[204] and more recently, Bunt et al's "Scheduling in Multiprogrammed Parallel Systems."[205] Lai and Hudak discuss "Memory Coherence in Shared Virtual Memory Systems."[206] Iftode and Singh survey different implementation strategies for maintaining memory coherence in "Shared Virtual Memory: Progress and Challenges."[207] A comprehensive survey, "Process Migration," has been published by Milojicic et al.,[208] and the corresponding problem of load balancing has been treated thoroughly by Langendorfer and Petri in "Load Balancing and Fault Tolerance in Workstation Clusters: Migrating Groups of Processes."[209] The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 1.
2. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htm>.
3. Breckenridge, C, "Tribute to Seymour Cray," November 19, 1996,<www.egl.ucsf.edu/home/tef/cray/tribute.html>.
4. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30, 1990, <www.businessweek.com/1989-94/pre88/b31571.htm>.
5. Breckenridge, C, "Tribute to Seymour Cray," November 19, 1996, <www.egl.ucsf.edu/home/tef/cray/tribute.html>.
6. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htrn>.

7. Breckenridge, C, "Tribute to Seymour Cray," November 19, 1996, <www.cgl.ucsf.edu/home/tef/cray/tribute.html>.
8. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htm>.
9. Top500, "TOP500 Supercomputer Sites: TOP500 List 06-2003," June 2003, <www.top500.org/list/2003/06/?page>.
1o. van der Steen, A., "TOP500 Supercomputer Sites: Intel Itanium 2," <www.top500.org/ORSC/2002/itaniurn.html>.
I I. Habata, S.; M. Yokokawa; and S. Kitawaki, "The Earth Simulator System," *NEC Research and Development,* January 2003, <www.nec.co.jp/techrep/en/r_and_d/r03/r03-nol/rd06.pdf >.
12. JAMSTEC, "Earth Simulator Hardware," <www.es.jamstec.go.jp/esc/eng/ES/hardware.html>.
13. Habata, S.; M. Yokokawa; and S. Kitawaki, "The Earth Simulator System," *NEC Research and Development,* January 2003, <www.nec.co.jp/techrep/en/r_and_d/r03/r03-nol/rd06.pdf >.
14. JAMSTEC, "Earth Simulator Hardware," <www.es.jamstec.go.jp/esc/eng/ES/hardware.html>.
15. JAMSTEC, "Earth Simulator Mission," <www.es.jamstec.go.jp/esc/eng/ESC/mission.html>.
16. "TOP500 Supercomputer Sites," June 1, 2003, <top500.org/lists/2003/06/l/>.
17. "Our Mission and Basic Principles," *The Earth Simulator Center,* October 17, 2003, <www.es.jamstec.go.jp/esc/eng/ESC/mission.html>.
18. Allison, D., "Interview with Seymour Cray," May 9,1995, <americanhistory.si.edu/csr/comphist/cray.htm>.
19. Breckenridge, C, "Tribute to Seymour Cray," November 19, 1996, <www.cgl.ucsf.edu/home/tef/cray/tribute.html>.
20. Allison, D., "Interview with Seymour Cray," May 9,1995, <americanhistory.si.edu/csr/comphist/cray.htm>.
21. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htrn>.
22. Breckenridge, C, "Tribute to Seymour Cray," November 19, 1996,<www.cgl.ucsf.edu/home/tef/cray/tribute.html>.
23. Allison, D., "Interview with Seymour Cray," May 9,1995, <americanhistory.si.edu/csr/comphist/cray.htm>.
24. Breckenridge, C, "Tribute to Seymour Cray," November 19, 1996,<www.cgl.ucsf.edu/home/tef/cray/tribute.html >.
25. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htrn>.
26. Allison, D., "Interview with Seymour Cray," May 9,1995, <americanhistory.si.edu/csr/comphist/cray.htm>.
27. Breckenridge, C, "Tribute to Seymour Cray," November 19, 1996, <www.cgl.ucsf.edu/home/tef/cray/tribute.html>.
28. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htm>.
29. Allison, D., "Interview with Seymour Cray," May 9,1995, <americanhistory.si.edu/csr/comphist/cray.htm>.
30. Breckenridge, C, "Tribute to Seymour Cray," November 19. 1996,<www.cgl.ucsf.edu/home/tef/cray/tribute.html>.
31. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htrn>.
32. Allison, D., "Interview with Seymour Cray," May 9,1995, <americanhistory.si.edu/csr/comphist/cray.htm>.
33. Breckenridge, C, "Tribute to Seymour Cray," November 19. 1996,<www.cgl.ucsf.edu/home/tef/cray/tribute.html>.
34. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htrn>.
35. Allison, D., "Interview with Seymour Cray," May 9,1995, <americanhistory.si.edu/csr/comphist/cray.htm>.
36. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htm>.
37. Mitchell, R., "The Genius: Meet Seymour Cray, Father of the Supercomputer," *BusinessWeek,* April 30,1990, <www.businessweek.com/1989-94/pre88/b31571.htrn>.
38. Cray Incorporated, "Historic Cray Systems," <www.cray.com/company/h_systems.html>.
39. Cray Incorporated, "Historic Cray Systems," <www.cray.com, company/h_systems.html>.
40. Allison, D., "Interview with Seymour Cray," May 9,1995, <americanhistory.si.edu/csr/comphist/cray.htm>.
41. Flynn, M., "Very High-Speed Computing Systems," *Proceedings of the IEEE,* Vol. 54, December 1966, pp. 1901-1909.
42. Flynn, M., and K. Rudd, "Parallel Architectures," *ACM Compiti- ■ ing* Surveys, Vol. 28, No. 1, March 1996, p. 67.
43. "Accelerating Digital Multimedia Production with Hyper-Threading Technology," January 15, 2003. <http://cedar.intel.com/cgi-bin/ids.dll/content/con¬tent.jsp?cntKey=Ceneric+Edito¬ri al%3a%3ahyperthreading_digitaljiultimedia&cntType=IDS_EDIT0RIAL&catCode=CDN&path=5>.
44. Flynn, M., and K. Rudd, "Parallel Architectures," *ACM Computing Surveys,* Vol. 28, No. 1, March 1996, p. 69.
45. Zhou, X, and K. Ross, "Research Sessions: Implementation Techniques: Implementing Database Operations Using SIMD Instructions," *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data,* June 2002, p. 145.
46. Flynn, M., and K. Rudd, "Parallel Architectures," *ACM Computing* Surveys, Vol. 28, No. 1, March 1996, pp. 68-69.
47. Hennessy, X, and D. Patterson, *Computer Organization and Design,* San Francisco, CA: Morgan Kaufmann, 1998, pp. 751-752.

48. Flynn, M., and K. Rudd, "Parallel Architectures," *ACM Computing Survey s,Vol.* 28, No. 1, March 1996, p. 68.

49. Gelenbe E., "Performance Analysis of the Connection Machine," *Proceedings of the 1990 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems,* Vol. 18, No. 1, April 1990, pp. 183-191.

50. Flynn, M., and K. Rudd, "Parallel Architectures," *ACM Computing* Surveys, Vol. 28, No. 1, March 1996, p. 69.

51. Crawley, D., "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," *UCL Image Processing Group Report 98/3,*1998, p. 3.

52. Crawley, D., "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," *UCL Image Processing Group Report 98/3,*1998, p. 3.

53. Bhuyan, L.; Q. Yang; and D. Agrawal, "Performance of Multiprocessor Interconnection Networks," *Computer,* Vol. 20, No. 4, April 1987, pp. 50-60.

54. Rettberg, R., and R. Thomas, "Contention is No Obstacle to Shared-Memory Multiprocessors," *Communications of the ACM,* Vol. 29, No. 12, December 1986, pp. 1202-1212.

55. Crawley, D., "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," *UCL Image Processing Group Report 98/3,*1998, p. 12.

56. Hennessy, J., and D. Patterson, *Computer Organization and Design,* San Francisco, CA: Morgan Kaufmann, 1998, p. 718.

57. Crawley, D., "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," *UCL Image Processing Group Report 98/3,*1998, p. 12.

58. Qin, X., and J. Baer, "A Performance Evaluation of Cluster Architectures," *Proceedings of the 1997 ACM SIGMETRICS International Conference on Measuring and Modeling of Computer Systems,* 1997, p. 237.

59. Friedman, M., "Multiprocessor Scalability in Microsoft Windows NT/2000," *Proceedings of the 26th Annual International Measurement Group Conference,* December 2000, pp. 645-656.

60. Enslow, P., "Multiprocessor Organization—A Survey," *ACM Computing Surveys,* Vol. 9, No. 1, March 1977, pp. 103-129.

61. Bhuyan, L.; Q. Yang; and D. Agrawal, "Performance of Multiprocessor Interconnection Networks," *Computer,* Vol. 20, No. 4, April 1987, pp. 50-60.

62. Stenstrom, P., "Reducing Contention in Shared-Memory Multiprocessors," *Computer,* Vol. 21, No. 11 November 1988, pp. 26-37.

63. Crawley, D., "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," *UCL Image Processing Group Report 98/3,*1998, p. 19.

64. "UltraSPARC-Ill," <www.sun.com/processors/UltraSPARC-III/USIIITech.html>.

65. Crawley, D., "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," *UCL Image Processing Group Report 98/3,*1998, p. 8.

66. Seitz, C, "The Cosmic Cube," *Communications of the ACM,* Vol. 28, No. 1, January 1985, pp. 22-33.

67. Padmanabhan, K., "Cube Structures for Multiprocessors," *Communications of the ACM,* Vol. 33, No. 1, January 1990, pp. 43-52.

68. Hennessy, J, and D. Patterson, *Computer Organization and Design,* San Francisco, CA: Morgan Kaufmann, 1998, p. 738.

69. Crawley, D., "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," *UCL Image Processing Group Report 98/3,*1998, p. 10.

70. "The nCUBE 2s," *Top 500 Supercomputer Sites,* February 16, 1998 <www.top500.org/0RSC/1998/ncube.html>.

71. Bhuyan, L.; Q. Yang; and D. Agrawal, "Performance of Multiprocessor Interconnection Networks," *Computer,* Vol. 20, No. 4, April 1987, pp. 50-60.

72. Crawley, D., "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," *UCL Image Processing Group Report 98/3,*1998, p. 15.

73. "Using ASCI White," October 7, 2002, <www.llnl.gov/asci/piatforms/white/>.

74. "IBM SP Hardware/Software Overview," February 26, 2003, <www.llnl.gov/computing/tutorials/ibmhwsw/#evolution>.

75. Skillicorn, D., "A Taxonomy for Computer Architectures," *Computer,* Vol. 21, No. 11, November 1988, pp. 46-57.

76. McKinley, D., "Loosely-Coupled Multiprocessing: Scalable, Available, Flexible, and Economical Computing Power," *RTC Europe,* May 2001, pp. 22-24, <www.rtceuropeonline.com/may2001/specialreport.pdf>.

77. McKinley, D., "Loosely-Coupled Multiprocessing: Scalable, Available, Flexible, and Economical Computing Power," *RTC Europe,* May 2001, pp. 22-24, <www.rtceuropeonline.com/may2001/specialreport.pdf>.

78. Enslow, P., "Multiprocessor Organization—A Survey," *Computing Surveys,* Vol. 9, No. 1, March 1977, pp. 103-129.

79. Van Lang.T, "Strictly On-Line: Parallel Algorithms for Calculating Underground Water Quality," *Linux Journal,* No. 63, July 1, 1999, <www.linuxjournal.com/article.php?sid=3021>.

80. Enslow, P., "Multiprocessor Organization—A Survey," *Computing* Surveys, Vol. 9, No. 1, March 1977, pp. 103-129.

81. Bartlett, J., "A NonStop Kernel," *Proceedings of the Eighth Symposium on Operating System Principles,* December 1981, pp. 22-29.

82. Enslow, P., "Multiprocessor Organization—A Survey," *Computing* Surveys, Vol. 9, No. 1, March 1977, pp. 103-129.

83. Wilson, A., "More Power to You: Symmetrical Multiprocessing Gives Large-Scale Computer Power at a Lower Cost with Higher Availability," *Datamation,* June 1980, pp. 216-223.

84. Rettberg, R., and R. Thomas, "Contention is No Obstacle to Shared-Memory Multiprocessors," *Communications of the ACM,* Vol. 29, No. 12, December 1986, pp. 1202-1212.

85. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 2.

86. Tripathi, A., and N. Karnik, "Trends in Multiprocessor and Distributed Operating System Design," *Journal of Supercomputing,* Vol. 9, No. 1/2,1995, p. 4.

87. Mukherjee, R; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 2.

88. Mukherjee, R; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, pp. 2-3.

89. Tripathi, A., and N Karnik, "Trends in Multiprocessor and Distributed Operating System Design," *Journal of Supercomputing,* Vol. 9, No. 1/2,1995, p. 3.

90. Joe, T, and J. Hennessy, "Evaluating the Memory Overhead Required for COMA Architectures," *Proceedings of the 21st Annual Symposium on Computer Architecture,* April 1994, p. 82.

91. Stenstrom, P.; T. and Joe; A. Gupta, "Comparative Performance Evaluation of Cache-Coherent NUMA and COMA Architectures," *Proceedings of the 19th Annual Symposium on Computer Architecture,* May 1992, p. 81.

92. Stenstrom, P.; T. Joe; and A. Gupta, "Comparative Performance Evaluation of Cache-Coherent NUMA and COMA Architectures," *Proceedings of the 19th Annual Symposium on Computer Architecture,* May 1992, p. 80.

93. Tripathi, A., and N. Karnik, "Trends in Multiprocessor and Distributed Operating System Design," *Journal of Supercomputing,* Vol. 9, No. 1/2,1995, p. 20.

94. Mukherjee, R; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, pp. 2-3.

95. Barroso, L.A.; J. Dean; and U. Hoelzle, "Web Search for a Planet: The Google Cluster Architecture," *IEEE MICRO,* March-April 2003, pp. 22-28.

96. Tripathi, A., and N. Karnik, "Trends in Multiprocessor and Distributed Operating System Design," *Journal of Supercomputing,* Vol. 9, No. 1/2,1995, p. 3.

97. Tanenbaum, A., and R. Renesse, "Distributed Operating Systems," *Computing Surveys,* Vol. 17, No. 4, December 1985, p. 420.

98. Lai, K., and P. Hudak, "Memory Coherence in Shared Virtual Memory Systems," *ACM Transactions on Computer Systems,* Vol. 7, No. 4, November 1989, p. 325.

99. Lai, K., and P. Hudak, "Memory Coherence in Shared Virtual Memory Systems," *A CM Transactions on Computer Systems,* Vol. 7, No. 4, November 1989, p. 325.

100. "Cache Coherence," *Webopedia.com,* <www.webopedia.com/TERM/C/cache_coherence.html>.

101. Stenstrom, P.; T. Joe; and A. Gupta, "Comparative Performance Evaluation of Cache-Coherent NUMA and COMA Architectures," *Proceedings of the 19th Annual Symposium on Computer Architecture,* May 1992, p. 81.

102. Soundararajan, V., et al., "Flexible Use of Memory for Replication/Migration in Cache-Coherent DSM Multiprocessors," *Proceedings of the 25th Annual Symposium on Computer Architecture,* June 1998, p. 342.

103. Lai, A. and B. Falsafi, "Comparing the Effectiveness of Fine-Grain Memory Caching against Page Migration/Replication in Reducing Traffic in DSM Clusters," *ACM Symposium on Parallel Algorithms and Architecture,* 2000, p. 79.

104. Soundararajan, V., et al., "Flexible Use of Memory for Replication/Migration in Cache-Coherent DSM Multiprocessors," *Proceedings of the 25th Annual Symposium on Computer Architecture,* June 1998, p. 342.

105. Baral, Y; M. Carikar; and P. Indyk, "On Page Migration and Other Relaxed Task Systems," *ACM Symposium on Discrete Algorithms,* 1997, p. 44.

106. Vergese, B., et al., "Operating System Support for Improving Data Locality on CC-NUMA Computer Servers," *Proceedings of the 7th International Conference on Architectural Support for Programming Languages and Operating Systems,* October 1996. p. 281.

107. Vergese, B., et al., "Operating System Support for Improving Data Locality on CC-NUMA Compute Servers," *Proceedings of the 7th International Conference on Architectural' Support for Programming Languages and Operating Systems,* October 1996. p. 281.

108. LaRowe, R.; C. Ellis; and L. Kaplan, "The Robustness of NUMA Memory Management," *Proceedings of the 13th ACM Symposium on Operating System Principles,* October 1991, pp. 137-138.

109. Lai, K., and P. Hudak, "Memory Coherence in Shared Virtual Memory Systems," *ACM Transactions on Computer Systems,* Vol. 7, No. 4, November 1989, pp. 326-327.

no. Lai, K., and P. Hudak, "Memory Coherence in Shared Virtual Memory Systems," *ACM Transactions on Computer Systems,* Vol. 7, No. 4, November 1989, pp. 327-328.

in. Tripathi, A., and N. Karnik, "Trends in Multiprocessor and Distributed Operating System Design," *Journal of Supercomputing.* Vol. 9, No. 1/2,1995, p. 21.

112. Iftode, L., and J. Singh, "Shared Virtual Memory: Progress and Challenges," *Proceedings of the IEEE Special Issue on Distributed Shared Memory,* Vol. 87, No. 3, March 1999, p. 498.

113. Iftode, L., and J. Singh, "Shared Virtual Memory: Progress and Challenges," *Proceedings of the IEEE Special Issue on Distributed Shared* Memory, Vol. 87, No. 3, March 1999, p. 499.

114. Gharacharloo, K., et al., "Memory Consistency and Event Ordering in Scalable Shared-Memory Multiprocessors," *Proceedings of the 17th Annual Symposium on Computer Architecture,* May 1990, pp. 17-19.

115. Carter, J; J. Bennett; and J. Zwaenepoel, "Implementation and Performance of Munin," *Proceedings of the 13th ACM Symposium on Operating System Principles,* 1991, pp. 153-154.

n6. Iftode, L., and J. Singh, "Shared Virtual Memory: Progress and Challenges," *Proceedings of the IEEE Special Issue on Distributed Shared Memory,\o\.* 87, No. 3, March 1999, pp. 499-500.

117. Dudnicki, C, et al., "Improving Release-Consistent Shared Virtual Memory Using Automatic Update," *Proceedings of the Second IEEE Symposium on High-Performance Computer Architecture,* February 1996, p. 18.

118. "Delayed Consistency and Its Effects on the Miss Rate of Parallel Programs," *Proceedings of Supercomputing '91,*1991, pp. 197-206.

119. Iftode, L,. and J. Singh, "Shared Virtual Memory: Progress and Challenges," *Proceedings of the IEEE Special Issue on Distributed Shared Memory,Vol.* 87, No. 3, March 1999, p. 500.

120. Tripath, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Designs," *Journal of Supercomputing,* Vol. 9, No. 1/2,1995, p. 14.

121. Tripath, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Designs," *Journal of Supercomputing,Vo\.* 9, No. 1/2,1995, p. 14.

122. Tripath, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Designs," *Journal of Supercomputing,Vol.* 9, No. 1/2,1995, p. 14.

123. Tannenbaum, A. S., and R. V. Renesse, "Distributed Operating Systems," *ACM Computing Surveys,* Vol. 17, No. 4, December 1985, p. 436.

124. Bunt, R. B.; D. T. Eager; and S. Majumdar, "Scheduling in Multiprogrammed Parallel Systems," *ACM SIGMETRICS,* 1988, p. 106.

125. BuntyR. B.; D. L. Eager; and S. Majumdar, "Scheduling in Multiprogrammed Parallel Systems," *ACM SIGMETRICS,* 1988, p. 105.

126. Bunt, R. B.; D. L. Eager; and S. Majumdar, "Scheduling in Multiprogrammed Parallel Systems," *ACM SIGMETRICS,* 1998, p. 106.

127. Gopinath, P., et al., "A Survey of Multiprocessing Operating Systems (Draft)," *Georgia Institute of Technology,* November 5, 1993, p. 15.

128. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993,1993, p. 15.

129. Bunt, R. B.; D. L. Eager; and S. Majumdar, "Scheduling in Multiprogrammed Parallel Systems," *ACM SIGMETRICS,* 1998, p. 106.

130. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 15.

131. Tripathi, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Design," *Journal of Supercomputing,* Vol. 9, No. 1/2,1995, p. 16.

132. Leutenegger, S. T, and M. K. Vernon, "The Performance of Multiprogrammed Multiprocessor Scheduling Policies," *Proceedings of the ACM Conference on Measurement and Modeling of Computer Systems,* 1990, p. 227.

133. Leutenegger, S. T., and M. K. Vernon, "The Performance of Multiprogrammed Multiprocessor Scheduling Policies," *Proceedings of the ACM Conference on Measurement and Modeling of Computer Systems,* 1990, p. 227.

134. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 17.

135. Tripathi, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Design," *Journal of Supercomputing,* Vol. 9, No. 1/2,1995, p. 16.

136. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 15.

137. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 16.

138. Gupta, A., and A. Tucker, "Process Control and Scheduling Issues for Multiprogrammed Shared-Memory Multiprocessors," *Proceedings of the 12th ACM Symposium on Operating System Principles,* 1989, p. 165.

139. Leutenegger, S. T, and M. K. Vernon, "The Performance of Multiprogrammed Multiprocessor Scheduling Policies," *Proceedings of the ACM Conference on Measurement and Modeling of Computer Systems,* 1990, pp. 227-228.

140. Carlson, B.; L. Dowdy; K. Dussa; and K-H. Park, "Dynamic Partitioning in a Transputer Environment," *Proceedings of the ACM,* 1990, p. 204.

141. Carlson, B.; L. Dowdy; K. Dussa; and K-H. Park, "Dynamic Partitioning in a Transputer Environment," *Proceedings of the ACM,* 1990, p. 204.

142. Carlson, B.; L. Dowdy; K. Dussa; and K-H. Park, "Dynamic Partitioning in a Transputer Environment," *Proceedings of the ACM,* 1990, p. 204.

143. Tripathi, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Designs," *Journal of Supercomputing,\o\.* 9, No. 1/2,1995, p. 17

144. Milojicic, D., et al, "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 241

145. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 246

146. Langendorfer, H, and S. Petri, "Load Balancing and Fault Tolerance in Workstation Clusters: Migrating Groups of Processes," *Operating Systems Review,* Vol. 29, No. 4, October 1995, p. 25

147. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 246

148. Eskicioglu, M., "Design Issues of Process Migration Facilities in Distributed Systems," *IEEE Computer Society Technical Committee on Operating Systems Newsletter,* Vol. 4, No. 2,1990, pp. 5-6.

149. Steketee, C; P. Socko; and B. Kiepuszewski, "Experiences with the Implementation of a Process Migration Mechanism for

Amoeba," *Proceedings of the 19th ACSC Conference,* January-February 1996.

150. Eskicioglu, M., "Design Issues of Process Migration Facilities in Distributed Systems," *IEEE Computer Society Technical Committee on Operating Systems Newsletter,* Vol. 4, No. 2,1990, pp. 5-6.
151. Milojicic, D., et. al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 255
152. Eskicioglu, M., "Design Issues of Process Migration Facilities in Distributed Systems," *IEEE Computer Society Technical Committee on Operating Systems Newsletter,* Vol. 4, No. 2,1990, p. 10.
153. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 256
154. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 21.
155. Steketee, C; P. Socko; and B. Kiepuszewski, "Experiences with the Implementation of a Process Migration Mechanism for Amoeba," *Proceedings of the 19th ACSC Conference,* January-February 1996.
156. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 259
157. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 256
158. Eskicioglu, M., "Design Issues of Process Migration Facilities in Distributed Systems," *IEEE Computer Society Technical Committee on Operating Systems Newsletter,* Vol. 4, No. 2,1990, p. 6.
159. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 256.
160. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 256
161. Eskicioglu, M., "Design Issues of Process Migration Facilities in Distributed Systems," *IEEE Computer Society Technical Committee on Operating Systems Newsletter,\o\.* 4, No. 2,1990, p. 7.
162. Douglis, E, and J. Ousterhout, "Transparent Process Migration: Design Alternatives and the Sprite Implementation," *Software Practice and Experience,* Vol. 21, No. 8, p. 764.
163. Zayas, E., "Attacking the Process Migration Bottleneck," *Proceedings of the Eleventh Annual ACM Symposium on Operating System Principles,* 1987, pp. 13-24.
164. Douglis, E, and J. Ousterhout, "Transparent Process Migration: Design Alternatives and the Sprite Implementation," *Software Practice and Experience,Vol.* 21, No. 8, p. 764.
165. Milojicic, D, et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 256
166. Eskicioglu, M., "Design Issues of Process Migration Facilities in Distributed Systems," *IEEE Computer Society Technical Committee on Operating Systems Newsletter,Vo\.* 4, No. 2,1990, p. 7.
167. Dougli, E, and J. Ousterhout, "Transparent Process Migration: Design Alternatives and the Sprite Implementation," *Software Practice and Experience,* Vol. 21, No. 8, p. 764.
168. Eskicioglu, M., "Design Issues of Process Migration Facilities in Distributed Systems," *IEEE Computer Society Technical Committee on Operating Systems Newsletter,* Vol. 4, No. 2,1990, pp. 6-7.
169. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5. 1993, p. 14.
170. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 14.
171. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5. 1993, pp. 14-15.
172. Diekmann, R.; B. Monien; and R. Preis, "Load Balancing Strategies for Distributed Memory Machines," *World Scientific,* 1997, p. 3.
173. Tannenbaum, A. S., and R. V. Renesse, "Distributed Operating Systems," *ACM Computing Surveys,* Vol. 17, No. 4, December 1985, p. 436.
174. Diekmann, R.; B. Monien; and R. Preis, "Load Balancing Strategies for Distributed Memory Machines," *World Scientific,* 1997, p. 5.
175. Diekmann, R.; B. Monien; and R. Preis, "Load Balancing Strategies for Distributed Memory Machines," *World Scientific,* 1997, p. 7.
176. Tannenbaum, A. S., and R. V. Renesse, "Distributed Operating Systems," *ACM Computing Surveys,* Vol. 17, No. 4, December 1985, p. 438.
177. Milojicic, D, et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 250.
178. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 251.
179. Shivarati, N; P. Kreuger; and M. Singhal, "Load Distributing for Locally Distributed Systems," *IEEE Computer,* 1992, pp. 37-39.
180. Milojicic, D., e. al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 251.
181. Shivarati, N.; P. Kreuger; and M. Singhal, "Load Distributing for Locally Distributed Systems," *IEEE Computer,* 1992, pp. 37-39.
182. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 251.
183. Shivarati, N; P. Kreuger; and M. Singhal, "Load Distributing for Locally Distributed Systems," *IEEE Computer,* 1992, pp. 37-39.
184. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 251.
185. Ahmad, I., and Y-K. Kwok, "Static Scheduling Algorithms for Allocating Directed Task Graphs to Multiprocessors," *Communications of the ACM,* 2000, p. 462.
186. Luling, R., et al., "A Study on Dynamic Load Balancing Algorithms," *Proceedings of the IEEE SPDP,* 1991, p. 686-689.
187. Luling, R., et al., "A Study on Dynamic Load Balancing Algorithms," *Proceedings of the IEEE SPDP,* 1991, p. 686-689.

[88. Tannenbaum, A. S., and R. V. Renesse, "Distributed Operating Systems," *ACM Computing Surveys,* Vol. 17, No. 4, December 1985, p. 438.

: S9. Tannenbaum, A. S., and R. V. Renesse, "Distributed Operating Systems," *ACM Computing Surveys,* Vol. 17, No. 4, December 1985, p. 438.

190. Barak, A., and A. Shiloh, "A Distributed Load Balancing Policy for a Multicomputer," *Software Practice and Experience,* December 15,1985, pp. 901-913.

191. Tripathi, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Designs," *Journal of Supercomputing,Vo*\. 9, No 1/2,1995, p. 11.

192. Mellor-Crummy, J. and M. Scott, "Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors," *ACM Transactions on Computer Systems,* Vol. 9, No. 1, February 1991, p. 22.

193. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 24.

194. Tripathi, A., and N M. Karnik, "Trends in Multiprocessor and Distributed Operating System Designs," *Journal of Supercomputing,Vol* 9, No 1/2,1995, pp. 11-12.

195. Tripathi, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Designs," *Journal of Supercomputing,* Vol. 9, No 1/2,1995, p. 12.

196. Mukherjee, B., and K. Schwan, "Experiments with Configurable Locks for Multiprocessors," *GIT-CC-93/05,* January 10,1993.

197. Tripathi, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Designs," *Journal of Supercomputing,* Vol. 9, No 1/2,1995, p. 13.

198. Mellor-Crummy, J. and M. Scott, "Scalable Reader-Writer Synchronization for Shared-Memory Multiprocessors," *Proceedings of the Third ACM SIGPLAN Symposium on Principles and Practices of Parallel Programming,* 1991, pp. 106-113.

199- Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 25.

200. Flynn, M., "Very High-Speed Computing Systems," *Proceedings of the IEEE,Vol.* 54, December 1966, pp. 1901-1909.

201. Crawley, D., "An Analysis of MIMD Processor Interconnection Networks for Nanoelectronic Systems," *UCL Image Processing Group Report 98/3,*1998, p. 3.

202. Mukherjee, B.; K. Schwan; and P. Gopinath, "A Survey of Multiprocessor Operating Systems," *GIT-CC-92/05,* November 5, 1993, p. 1.

203. Tripathi, A., and N. Karnik, "Trends in Multiprocessor and Distributed Operating System Design," *Journal of Supercomputing,* Vol. 9, No. 1/2,1995, p. 3.

204. Leutenegger, S. T., and M. K. Vernon, "The Performance of Multiprogrammed Multiprocessor Scheduling Policies," *Proceedings of the ACM Conference on Measurement and Modeling of Computer Systems,* 1990, p. 227.

205. Bunt, R. B.; D. L. Eager; and S. Majumdar, "Scheduling in Multiprogrammed Parallel Systems," *ACM SIGMETRICS,* 1988, p. 106.

206. Lai, K., and P. Hudak, "Memory Coherence in Shared Virtual Memory Systems," *ACM Transactions on Computer Systems,* Vol. 7, No. 4, November 1989, p. 325.

207. Iftode, L., and J. Singh, "Shared Virtual Memory: Progress and Challenges," *Proceedings of the IEEE Special Issue on Distributed Shared Memory,* Vol. 87, No. 3, March 1999, p. 498.

208. Milojicic, D., et al., "Process Migration," *ACM Computing Surveys,* Vol. 32, No. 3, September 2000, p. 241

209. Langendorfer, H., and S. Petri, "Load Balancing and Fault Tolerance in Workstation Clusters: Migrating Groups of Processes," *Operating Systems* Review, Vol, 29, No. 4, October 1995, p. 25

Networking and Distributed Computing

Whatever shall we do in that remote spot?
—Napoleon Bonaparte —

Part 6

With the popularization of the Web in 1993, Internet usage literally exploded. Now there is a huge focus on building distributed, Internet-based applications; this is profoundly affecting operating systems design. Chapter 16 introduces computer networking and discusses network topologies and types, and the client/server networking model. We carefully explain the four layers of the Internet's TCP/IP protocol stack. Chapter 17 introduces distributed systems and discusses attributes, communication, synchronization, mutual exclusion and deadlock. The chapter also presents case studies of the Sprite and Amoeba distributed operating systems. Chapter 18 discusses distributed file systems, clustering, peer-to-peer distributed computing, grid computing, Java distributed computing technologies and the emerging technology of Web services.

The humblest is the peer of the most powerful.
—John Marshall Harlan—

Live in fragments no longer. Only connect.

—Edward Morgan Forster—

What networks of railroads, highways and canals were in another age, the networks of telecommunications, information and computerization...are today.

—Bruno Kreisky—

It took five months to get word back to Queen Isabella about the voyage of Colombus two weeks for Europe to hear about Lincoln's assassination, and only 1.2 seconds to get the words from Neil Armstrong that man can walk on the moon.

—Isaac Asimov—

Chapter 16

Introduction to Networking

Objectives

After reading this chapter, you should understand:

- *the central role of networking in today's computer systems.*
- *various network types and topologies.*
- *the TCP/IP protocol stack.*
- *the capabilities of TCP/IP's application, transport, network and link layers.*
- *protocols such as HTTP, FTP, TCP, UDP, XCP, IP and IPv6.*
- *network hardware and hardware protocols such as Ethernet and Wireless 802.11.*
- *the client/server networking model.*

Chapter Outline

16.1 Introduction

16.2 Network Topology

Mini Case Study: Symbian OS

16.3 Network Types

16.4 TCP/IP Protocol Stack

16.5 Application Layer

16.5.1 Hypertext Transfer Protocol (HTTP)

16.5.2 File Transfer Protocol (FTP)

16.6 Transport Layer

Anecdote: Unreliable Communication Lines Can Lead to Embarrassment

16.6.1 Transmission Control Protocol (TCP)

16.6.2 User Datagram Protocol (UDP)

16.7 Network Layer

16.7.1 Internet Protocol (IP)

16.7.2 Internet Protocol version 6 (IPv6)

16.8 Link Layer

16.8.1 Ethernet

16.8.2 Token Ring

16.8.3 Fiber Distributed Data Interface (FDDI)

16.8.4 IEEE 802.11 (Wireless)

16.9 Client/Server Model

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

16.1 Introduction

Networks have become almost as important as the computers they connect, enabling users to access resources that are available on remote computers and communicate with other users around the world. Talking on the telephone, watching cable television, using a cellular phone, making a credit card purchase, withdrawing money from an ATM, browsing the Web and sending e-mail are all activities that rely on networked computers. As users, we have come to expect that network communication will occur quickly and without error.

This chapter discusses common network layouts, focusing on how **hosts**—entities that receive and provide services over a network—are connected by **links**—media over which network services are transmitted. If a link breaks, a host fails or a message is lost, network communication can be interrupted. We introduce the **TCP/IP protocol stack,** which provides well-defined interfaces to enable communication between computers across a network and to allow problems to be fixed as they arise. The TCP/IP protocol stack splits networked communication into four logical levels called **layers.** Each layer provides functionality for the layers above it to ease the development, management and debugging of networks and simplify the programming of applications that rely on those networks. A layer is implemented by following certain **protocols**—sets of rules that govern how two entities should interact. In our discussion of each layer, we consider popular Internet protocols that allow users worldwide to communicate. We conclude by discussing the popular client/server model of network communication.

The concepts presented in this chapter will help you understand the chapters on distributed systems (Chapter 17 and Chapter 18), security (Chapter 19) and the case studies on Linux (Chapter 20) and Windows XP (Chapter 21).

Self Review

1. Why does the TCP/IP protocol stack separate network communication into four layers?
2. What problems can arise during network communication?

Ans: **1)** Separating network communication into four layers modularizes the communication. Developers can focus on one layer at a time. This eases development, management and debugging of networks and networked applications. **2)** Hosts can fail, links can break and messages can be lost.

16.2 Network Topology

Network topology describes the relationship among the hosts, also called **nodes,** on a network. A **logical topology** displays which nodes in a network are directly connected (i.e., which nodes can communicate with each other without relying on any intermediate nodes).[1] Common network topologies (Fig. 16.1) include bus, ring, star, tree, mesh and fully-connected mesh networks.

Nodes on a **bus** (or **linear) network** (Fig. 16.1, part a) are all connected to a single, common communication link (called a bus). Bus networks are simple

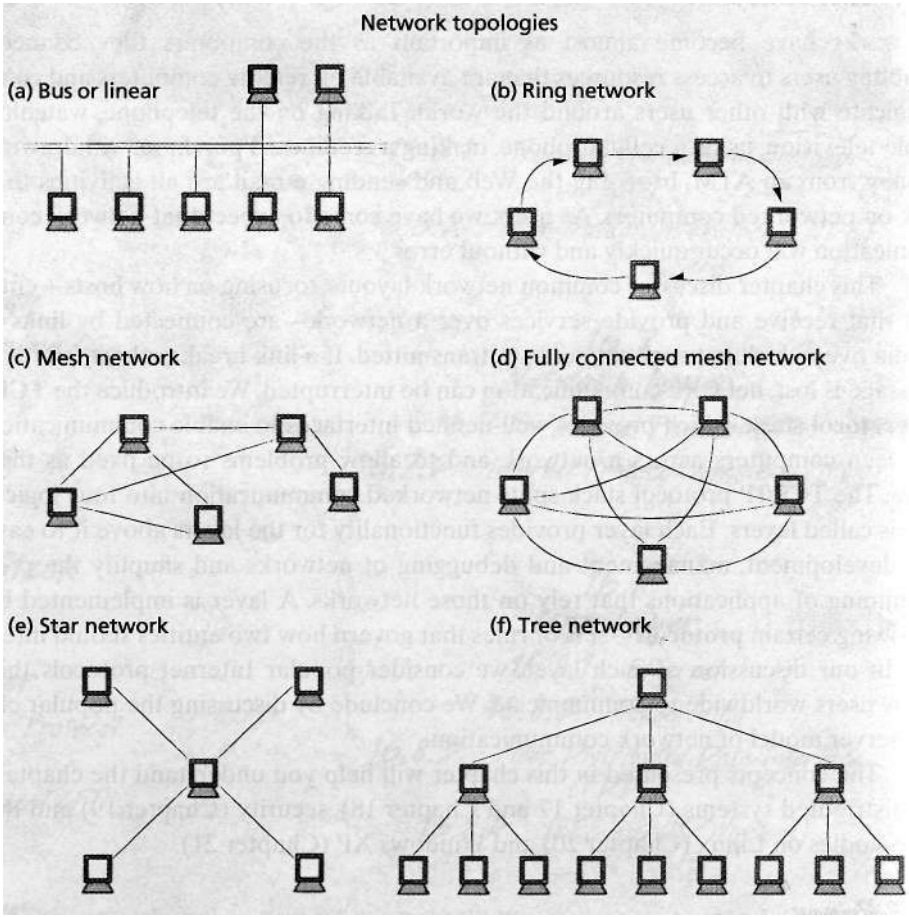


Figure 16.1 | *Network topologies.*

because they do not require intermediate nodes to forward messages to other nodes. As messages travel along a link, resistance in the medium causes the signal to lose strength—this is known as **attenuation.** Since bus networks do not have intermediate nodes that retransmit messages, the length of the bus communication medium must be limited to minimize attenuation. If any individual node in a bus network fails, the entire network will continue functioning. If the bus itself fails, the entire network will fail. Bus networks are appropriate for homes and small offices. '

Ring networks (Fig. 16.1, part b) consist of a set of nodes, each of which maintains exactly two connections to other nodes such that a message sent through one connection can eventually return via the other. Ring networks can grow to be larger than bus networks, because each node in the ring forwards each message; this limits message attenuation but introduces a retransmission delay (i.e., the time required for a node to process a message before retransmitting it). One of the most significant limitations of a ring network is that if one node in a ring fails, then communica-

tion in the entire ring will fail. This means that a ring network has limited fault tolerance, since the network cannot recover from a single node failure.

Star networks (Fig. 16.1, part e) contain a single central node, or **hub,** that is connected to all of the other nodes in the network and is responsible for relaying messages between nodes. All transmissions in a star network pass through the hub. Since communications over star networks go through a single intermediate node, attenuation will limit the geographical size of the network, but transmission delay is smaller than in ring networks. The network can survive the failure of one of the outer nodes, but the entire network will fail if the central hub fails. Since the central hub controls all communication, a bottleneck will occur if the network demand exceeds the processing capabilities of the hub.

Tree networks (Fig. 16.1, part f) are hierarchical networks that consist of a root node and several subnodes, called children, that can have subnodes of their own. A tree network can be viewed as multiple star networks. The hub of the first star network is the root of the tree. Each child node in this star serves as a hub for another star network. Hubs are responsible for relaying information to the nodes in their immediate networks. A tree topology is often used to join nodes that communicate with each other frequently, thereby increasing network efficiency.[6, 7]

In **mesh networks** (Fig. 16.1, part c), at least two nodes have more than one path connecting them. A mesh network in which each node is directly connected to every other node is a **fully-connected mesh network** (Fig. 16.1, part d). Mesh networks and fully connected networks are the most fault-tolerant topologies, because typically there are multiple paths between each pair of nodes. The primary disadvantage of mesh networks is the complexity associated with directing messages between nodes that have no direct connection. With fully-connected networks, this is simple since each pair of nodes has a direct link between them. The problem with fully-connected networks is that as the number of nodes increase, the number of links to connect those nodes increases exponentially.[8,9]

The proliferation of wireless network technology has introduced **ad hoc networks** (see the Mini Case Study, Symbian OS). An ad hoc network is spontaneous—any number of devices may be connected to it at any time. These devices become part of, and leave, the network at random. Ad hoc networks consist of any combination of wireless and wired devices. The network topology can change rapidly, which makes it difficult for the network to be governed by a central node. The variable nature of ad hoc networks makes determining their topology a challenging problem in current research.[10]

Self Review

1. Why would a network for a mission-critical system (e.g., a nuclear power plant or an air traffic control system) not be built with a ring topology?
2. Why do wireless devices require a spontaneous network?

Ans: **1)** A mission-critical system would not use a ring topology because of the lack of fault tolerance. If a single node failed, the entire network would fail. Mission-critical systems

require networks with multiple levels of redundancy to ensure continuous operation. **2)** The nature of wireless devices is that as they move from place to place; they join and exit multiple networks. Ad hoc networks do not require a fixed network topology.

16.3 Network Types

The two most prevalent types of networks are **local area networks (LANs)** and **wide area networks (WANs).** A LAN is a network with limited geographic range designed to optimize data transfer rates between its nodes. Local area networks interconnect resources using high-speed communication paths with network protocols optimized for local area environments such as office buildings or college campuses. LANs benefit by independence from the larger networks. Those that are company or university owned can be upgraded or reorganized at the company or university's discretion. LANs are also free from the congestion that can arise in

Mini Case Study

Symbian OS

Symbian (www.symbian.com), maker of the Symbian operating system, was founded in 1998 by mobile phone manufacturers Ericsson, Nokia, Motorola and Psion to develop a cross-platform mobile phone operating system to replace their individual, incompatible OSs.[11, 12] The result is Symbian OS, a small operating system that runs on "smart phones"[13]—mobile phones with the functionality of a PDA (Personal Digital Assistant). Symbian OS is unique in that it was built for mobile phones,[14,15] while its major competitors such as Windows CE, Linux and PalmOS were all originally designed for different systems, then adapted to mobile phones.

Symbian focuses on size and time efficiency.[16]-[17] The Symbian OS complies with open standards such as the POSIX API[18]—the IEEE's Portable Operating System Interface standard—as well as APIs for high-performance graphics and multimedia, mobile browsing and messaging,, communications and mobile telephony, security and data management, to make it easier for application developers to write compatible software.[19, 20] Also, since Java is becoming standard on mobile phones and has a large application developer base, an implementation of the Java mobile run-time environment (MIDP) is included.[21]-[22]

Symbian's latest release, v7.0s, is intended for third-generation (3G) mobile phones and comes equipped with new functionality to meet the needs of these high-performance wireless devices. Some of the new features of OS v7.0s include a multithreaded framework for multimedia, Java Wireless Messaging 1.0 and Bluetooth 1.1 capability.[23] OS v7.0s also supports W-CDMA (wideband code-division multiple access), which enables 3G mobile phones to transfer data at much higher rates, up to 2Mbps.[24] These new features are built upon the extensive collection of APIs common in all recent Symbian OS releases.

lager networks that must serve millions of users. Since LANs are autonomous, they can be customized to meet the needs of a particular group and can employ any of the network topologies discussed in Section 16.2.[25]

WANs are broader, consisting of many LANs and connecting many computers over great distances; the largest WAN by far is the Internet. WANs generally employ a mesh topology, operate at lower speeds than LANs and have higher error rates because they must interact with multiple, often heterogeneous LANs and WANs.

Self Review

1. What is the purpose of creating smaller subnetworks within a larger network, such as a LAN that comprises a piece of a WAN?
2. What are the disadvantages of WANs compared to LANs?

Ans: **1)** A subnetwork, such as a LAN on a college campus, allows for a group of related computers to be directly connected for faster transmission, higher capacity, greater management flexibility and customization. **2)** WANs often operate at lower speeds and have higher error rates than LANs because they must interact with multiple, often heterogeneous LANs and WANs.

16.4 TCP/IP Protocol Stack

The four layers of networked communication as defined by the TCP/IP protocol stack are the application, transport, network and link layers. The **application layer** is the highest level and provides protocols (e.g., HTTP, FTP) for applications, such as Web browsers and Web servers, to communicate with each other. The **transport layer** is responsible for end-to-end communication of information from the sending process to the receiving process. The **network layer** is responsible for moving the data from one computer to the next (also known as routing). The transport layer relies on the network layer to determine the proper path from one end of the communication to the other. The **link layer** translates information between bits and a physical signal that travels through the physical link (e.g., a network cable).[27]

As a message travels down the stack, each layer receives the message and adds **control information** to the beginning of the message (called a **header)** or to the end of the message (called a **trailer)** to enable communication with the corresponding layer on other hosts. The control information might include, for example, the addresses of the source and destination hosts, or the type or size of data that is being sent. This new message then is passed to the layer below it (or to the physical medium if the message is coming from the link layer). When receiving information over a network, each layer receives data from the layer below it (or from the physical medium in the case of the link layer). The layer strips off the control information from the corresponding layer on the remote host. This data is often used to ensure that the message is valid and directed to the current host. The message then is passed to the layer above it (or to the process if the message is being passed upward by the application layer). We discuss these layers in detail in the sections that follow.

In 1984, the International Organization for Standardization (ISO) introduced the Open Systems Interconnection (OSI) Reference Model to establish an international standard for communication between applications over a network. Although not followed strictly on the Internet, it remains an important model to understand. The OSI protocol stack consists of the four layers we discussed previously plus three additional ones. The seven layers of the OSI model are application, presentation, session, transport, network, data link and physical.

The application, presentation and session layers in OSI correspond to the application layer in the TCP/IP protocol stack. In the OSI model, the **application layer** interacts with the applications and provides network services, such as file transfer and e-mail. The **presentation layer** solves certain compatibility problems, such as when the two end users use different data formats, by translating the application data into a standard format that can be understood by other layers. The **session layer** establishes, manages and terminates the communication between two end users. The transport and network layers correspond with the transport and network layers of the TCP/IP protocol stack, respectively. The data link layer and physical layer in OSI correspond to the link layer in the TCP/IP protocol stack. At the sender, the **data link layer** converts the data representation it receives from the network layer into bits and at the destination, it converts bits received into the data representation for the network layer. The **physical layer** transmits bits over the physical medium, such as cables.[28, 29, 30, 31]

Self Review

1. What type of information is placed in the header or trailer of a message?
2. Briefly list the capabilities of each of the four layers of the TCP/IP protocol stack.

Ans: **1)** Control information is placed in the header or trailer of a message to govern the communication between the two hosts; this information can include the addresses of the two hosts, or the type or size of the data. **2)** The application layer allows applications on remote hosts to communicate with one another. The transport layer is responsible for end-to-end communication between two hosts. The network layer is responsible for sending a packet to the next host toward the destination. The link layer serves as an interface between the network layer and the physical medium through which the information travels.

16.5 Application Layer

The application layer provides a well-defined interface for applications on different computers to communicate with one another; for example opening a remote file requesting a Web page, transferring an e-mail or calling a remote procedure. Application layer protocols simplify communication between processes on a network and determine how processes should interact.[32] Common protocols of the application layer include Hypertext Transfer Protocol (HTTP), File Transfer Protocol (FTP), Simple Mail Transfer Protocol (SMTP), Domain Name System (DNS) and Secure Socket Shell (SSH).

Many application layer protocols interact with resources on remote hosts. These resources are specified by a **Uniform Resource Identifier (URI)**, which is a name that references a specific resource on a remote host. The more common term **Uniform Resource Locator (URL)** describes URIs that access resources in common protocols such as HTTP or FTP.[33] Most URLs contain the protocol, host, port and path of the resource. The protocol is the application layer protocol that is being used to communicate (e.g., HTTP or FTP). The host is the fully qualified name of the host computer. We discuss computer naming conventions in Section 16.7.1, Internet Protocol (IP). The port determines the socket to which a message should be passed—a socket is a software construct that represents one endpoint of a connection. Processes use sockets to send and receive messages over a network. The path is the location of the resource on that host.

Consider the URL http://www.deitel.com/index.html. The first part, `http://`, denotes that this URL is for HTTP. The second part, www.deitel.com, is the host name. Specifically, it accesses the company Web site for Deitel & Associates. The third part of the URL, the port, is omitted. Certain ports have been assigned by the Internet Assigned Numbers Authority as "well-known" ports; these ports are used by common application layer protocols. For example, HTTP commonly uses port 80, whereas FTP commonly uses ports 20 and 21.[34] By omitting the port, this URL connects to port 80, the default port for HTTP. The final part of the URL, `index.html`, specifies the path and name of the resource.

Self Review

1. Name some common protocols of the application layer.
2. Why is the port an important part of a URL?

Ans: **1)** FTP, SMTP, DNS, SSH and HTTP. **2)** The port identifies the socket on the computer to which the message should be passed.

16.5.1 Hypertext Transfer Protocol (HTTP)

The **Hypertext Transfer Protocol (HTTP)** is a versatile application layer protocol, While HTTP is commonly used to transmit HTML documents over the Internet, it allows the transfer of data in several formats using **Multipurpose Internet Mail Extensions (MIME).** MIME defines five content types: *text, image, audio, video* and *application.* The first four types are often used in multimedia Web pages; *application* is generally reserved for transferring binary files.

HTTP consists of requests for resources and responses from remote hosts. An **HTTP request** involves an action and a resource's URI. The action specifies the operation to be performed on the resource. The remote host processes the request and replies with an **HTTP response,** which contains in its header a code that tells the client whether the request was processed correctly or an error occurred. If the request was processed correctly, the requested resource is returned along with the

header. The header also specifies the MIME type of the resource to notify the requesting application about the type of content it is receiving.[35, 36, 37]

SelfReview

1. What would happen if a client sent an HTTP request for a resource that did not exist?
2. What is the purpose of MIME types?

Ans: **1)** The server would reply with an HTTP response that included an error code in the header. **2)** The MIME type provides the client with knowledge about the type of data being transmitted. This aids the client in determining how to process the data.

16.5.2 File Transfer Protocol (FTP)

File Transfer Protocol (FTP) is an application layer protocol that allows the transfer of files between remote hosts. FTP specifies connections between two ports: one port (typically port 21) sends control information that governs the session; the other port (typically port 20) sends the actual data. After a connection is established, the client specifies actions for the FTP server to perform by issuing various commands to the server (Fig. 16.2). The server attempts to satisfy each command, then issues a response with the result.[38]

Sefffovtetf

1. What ports do FTP hosts usually use to communicate?
2. What is the purpose of FTP?

Ans: **1)** FTP uses two ports to communicate, typically ports 20 and 21. **2)** FTP allows the transfer of files between hosts.

16.6 Transport Layer

The transport layer is responsible for the end-to-end communication of data between hosts. It receives data from the application layer, breaks it into smaller

Name	***Function***
CDUP	Change from the current directory to the parent of the current directory
CWD	Change the working directory.
PWD	Print the path of the working directory.
LIST	List the contents of the working directory.
DELE	Delete the specified file.
RETR	Retrieve the specified file.
STOR	Upload the specified file.
QUIT	Terminate the FTP session.

Figure 16.2 | *FTP commands.*

pieces suitable for transport, appends control information to these pieces and sends them to the network layer.

There are two primary approaches to implementing the transport layer: **coll--ection oriented** and **connectionless.** Connection-oriented services are modeled after the telephone system, in which a connection is established and held for the length of the session. Connectionless services are modeled after the postal service, in which two letters mailed from one place to the same destination may actually take two dramatically different paths through the system and even arrive at different times or not at all.

In a connection-oriented approach, hosts send each other control information — through a technique called **handshaking—**to initiate an end-to-end connection. Many networks are **unreliable,** which means that data sent across them may be damaged or lost (see the Anecdote, Unreliable Communication Lines Can Lead to Embarrassment). These networks do not guarantee anything about the data sent; it could arrive corrupted or out of order, as duplicates or not at all. These networks

Anecdote

Unreliable Communication Lines Can Lead to Embarrassment

This one happened to one of the authors, HMD. In the early 1980s, he was offered a consulting opportunity. To formalize the relationship, he had to send the client a detailed resume along with some other materials by overnight package courier. The package pickup deadline was approaching. HMD quickly updated his resume using a terminal at his home connected by an old-style acoustic coupler to phone lines and in turn to a time-sharing computer at the university where he was a professor at the time. He typed, "Since 1978, Harvey M. Deitel has been an assistant professor at Boston University" and started the printout (which was done on an old-style, 30-character-per-second dot matrix printing terminal). He walked into another room. The package courier came. He asked his wife to please tear off the printout, throw it in the package, seal it and give it to the courier. She said, "Harvey, I don't think you want to send this; you'd better take a look at it." HMD came back into the room. Data transmission over phone lines was a bit unreliable at the time. The printer failed and stopped typing at a most unfortunate point in the middle of the sentence (figure it out)! If HMD had sent the package, he surely would not have been given the consulting job! This was a humbling experience!

Lesson to operating systems designers: You can't design in a vacuum and simply assume that parts of a system "you're not responsible for" will indeed work perfectly. Every part of a system can fail—the software for which operating systems designers and implementors are most directly responsible, the hardware, the communications lines and even the people who work with the system as users or administrators.

make only a "best effort" to deliver data. A connection-oriented approach ensures reliability on unreliable networks. These protocols make communication **reliable.** guaranteeing that sent data will arrive at the intended receiver undamaged and in the correct sequence. Hosts might also exchange information to regulate the termination of the session so that no processes are left running on either host.

In a connectionless approach, the two hosts do not handshake before transmission and reliability is not guaranteed—data sent may never reach the intended recipient. A connectionless approach, however, avoids the overhead associated with handshaking and enforcing reliability; less information often needs to be passed between the hosts.[39, 40, 41, 42, 43]

Self Review

1. Why would a bank not implement a connectionless transport layer?
2. Most streaming media applications require a connectionless transport layer. What are some possible reasons for this?

Ans: **1)** A bank performs business-critical transactions that must be carried out correctly. A connectionless transport layer could lose transactions. **2)** Streaming media applications do not require the reliability of a connection-oriented approach, but benefit by the smaller amount of control information that comes with a connectionless approach.

16.6.1 Transmission Control Protocol (TCP)

The **Transmission Control Protocol (TCP)** is a connection-oriented transmission protocol that guarantees that data (called **segments** in TCP) sent from a sender will arrive at the intended receiver undamaged and in the correct sequence. Both HTTP and FTP rely on TCP to guarantee reliable communication. TCP handles error control, congestion control and retransmission, allowing protocols like HTTP and FTP to send information across a network as simply and reliably as writing to a file on a local computer.

To set up a connection, TCP uses a three-way handshake. One of the purposes of this handshaking is to synchronize sequence numbers between the two hosts. The sequence number of each host increases by one with each segment sent and is placed in each segment's header. The destination host uses these sequence numbers to rearrange the segments if they are received out-of-order. First, the source host sends a **synchronization segment (SYN)** to the destination host. This segment requests that a connection be made and contains the sequence number of the source host. The destination host responds with a **synchronization/acknowledgement segment (SYN/ACK)** which is an acknowledgement of the connection being established and contains the sequence number for the destination host. Finally, the source host responds with an **acknowledgement segment (ACK)** which finalizes the connection.[44]

When the destination host receives a segment, it responds with an acknowledgement (ACK) containing the sequence number of the received segment. If the source host does not receive an ACK for a particular sequence number, it will resend the segment with that sequence number after a certain wait time. This pro-

cess guarantees that all segments will eventually be received by the destination host, duplicates will be discarded and the segments will be reassembled, if necessary, into the original order.[45, 46, 47, 48, 49]

In addition to reliability, TCP offers flow control and congestion control. **Flow control** regulates the number of segments sent by each host in an attempt not to overwhelm the receiver of those segments. **Congestion control** restricts the number of segments sent from a single host in response to overall network congestion. TCP implements both flow control and congestion control by maintaining a **TCP window** for the sender and receiver. The sender cannot send more segments than specified by the window before receiving an ACK from the receiver. The receiver calculates and sends its window along with each ACK that it sends. There are tradeoffs associated with the size of the window advertised. A large window leads to the transmission of many segments. If the network or the receiver cannot handle this volume of segments, some will be discarded leading to retransmission of segments and inefficiency. Smaller windows can reduce throughput and result in the network being underutilized.

Self Review

1. TCP guarantees that segments will be delivered to the application in order, but the segments may not arrive at the host in order. What additional resource must TCP use?
2. What is the difference between flow control and congestion control?

Ans: **1)** TCP must possess a buffer to hold any segments received out of order until the missing segments are received. **2)** Flow control deals with restricting the number of segments sent by the source host so as to not overwhelm the destination host. Congestion control restricts the number of segments sent from a single host to help restrict network congestion.

16.6.2 User Datagram Protocol (UDP)

Applications that do not require the reliable end-to-end transmission guaranteed by TCP may be better served by the connectionless **User Datagram Protocol (UDP).** UDP incurs the minimum overhead necessary to implement the transport layer. There is no guarantee that UDP segments, called **datagrams,** will reach their destination or arrive in their original order.

There are benefits to using UDP over TCP. UDP has little overhead because UDP headers are small; they do not need to carry the information TCP carries to ensure reliability. UDP also reduces network traffic relative to TCP due to the absence of ACKs, handshaking, retransmissions, etc.

Unreliable communication is acceptable in many situations. First, reliability is not necessary for some applications, so the overhead imposed by a protocol that guarantees reliability can be avoided. Second, some applications, such as streaming audio and video, can tolerate occasional datagram loss. This usually results in a small pause (or "hiccup") in the audio or video being played. If the same application were run over TCP, a lost segment could cause a significant pause, since the protocol would wait until the lost segment was retransmitted and delivered correctly before continu-

ing. Finally, applications that need to implement their own reliability mechanisms different from that provided by TCP can build such mechanisms over UDP.[50, 51, 52]

Self Review

1. If a UDP datagram is lost due to network traffic, how does the host that sent the datagram respond?
2. Why would an application choose to use UDP as its transport layer protocol?

Ans: **1)** The host does nothing. UDP includes no means by which the sending host could learn that a datagram was lost. The host does not retransmit. **2)** An application would use UDP if reliability were not necessary (e.g., as in streaming audio and video applications) or if the application needed to implement its own reliability mechanisms.

16.7 Network Layer

The network layer receives data from the transport layer and is responsible for sending the data (called **datagrams)** to the next stop toward the destination through a process known as routing. **Routing** is the two-step process of determining the next host for a datagram toward the destination and sending the datagram along this path. **Routers** are computers that connect networks. Note that a router determines the next host for a given source and destination given its particular picture of the network at the time. Networks can change quickly and routers cannot determine these changes instantaneously, so a router's knowledge of the network is not always complete and up-to-date.

Routers determine the next host for a given datagram based on information such as network topologies and link quality, which includes strength of signal, error rate and interference. This information is propagated throughout networks using various routing protocols, one of the better known being the **Routing Information Protocol (RIP)—**an application layer protocol operating over UDP. RIP requires each router in a network to transmit its entire **routing table** (a hierarchical matrix listing the current network topology) to its closest neighboring routers. The process is continued until all routers are aware of the current topology. As networks grow in size, each router needs to keep track of the entire network topology This generates excessive network traffic in large networks, which is why RIP is used almost exclusively on smaller networks.

Routers keep queues to manage datagrams, because it takes time for the router to determine the best route and send the datagram. When a datagram arrives from a network, it is placed into a queue until it can be serviced. Once the queue is full, additional datagrams are simply dropped from the network. It is therefore important to make sure senders do not overwhelm routers. Transport layer protocols must try not to overrun the queues.

Unlike the transport layer, which is concerned with sending data from the originating host to the destination host, the network layer at a particular host sends datagrams only to the next host toward the destination host. This process continues until the datagram reaches its destination host.

Self Review

1. What is the major drawback to the Routing Information Protocol that causes it to be reserved for smaller networks?
2. What happens when network congestion causes a router's queue to fill up with unserviced datagrams?

Ans: **1)** Each router in a network is required to transmit its entire routing table to its closest neighbor, regardless of whether or not changes have occurred. In large networks, these routing tables are larger, which causes increased network traffic. **2)** When a router's queue is full, datagrams that arrive subsequently are discarded. Hosts using TCP will consider this an example of network congestion and slow down transmission.

16.7.1 Internet Protocol (IP)

The **Internet Protocol (IP)** is the dominant network layer protocol for transmitting information over a network. IP allows smaller networks. (LANs and small WANs) to be combined into larger networks (WANs, such as the Internet). IP version 4 (IPv4) is the version currently used by most networks. Destinations on the Internet are specified by **IP addresses,** which are 32-bit numbers in IPv4. Thus, there are 2^{32}, or approximately 4 billion, unique IP addresses. IP addresses are generally divided into four octets (8-bit bytes). An 8-bit byte can represent a decimal number from 0 to 256.

One or more host names are mapped to an IP address through the **Domain Name System (DNS).** When a person enters a host name into a browser, it uses DNS to find the correct IP address for that host name. In this way, an easily remembered name is established as an alias for the IP address. This IP address is then used by the transport and network layers to deliver the data.[53, 54, 55]

Self Review

1. How many possible IPv4 addresses are there?
2. How does a Web browser convert a host name into an IP address?

Ans: **1)** There are 2^{32}, or approximately 4 billion, unique IP addresses. **2)** The Web server uses the Domain Name System (DNS) to convert a host name into an IP address.

16.7.2 Internet Protocol version 6 (IPv6)

Every device that accesses the Internet must be assigned an IP address. As the number of devices (e.g., PDAs and cellular phones) with network access increases, the pool of available IP addresses shrinks. In the near future, the number of remaining 32-bit IP addresses will run out. To combat this problem, the Internet Engineering Task Force has introduced **Internet Protocol version 6 (IPv6).** IPv5 was proposed in 1990, nine years after IPv4, but it was never implemented. IPv6 eliminates many limitations of IPv4. IPv6 addresses are 128 bits long, yielding 2^{128} (approximately 3.4×10^{38}) possible addressable nodes.

IPv6 specifies three types of addresses: unicast, anycast and multicast. A **unicast address** allows users to send a datagram to a single host. **Anycast addresses allow** users to send a datagram to any one of a group of hosts. For example, a series

of routers that provide access to a LAN could be assigned anycast addresses. Incoming datagrams would be delivered to the nearest router with that address. **Multicast addresses** allow users to send datagrams to all hosts in a group.

IPv6's headers are simpler than those of IPv4, because several of the header fields have been eliminated or offered as options in IPv6. This increases the speed with which datagrams are processed. Fewer restrictions are imposed on the header format to increase extensibility; IPv6 provides mechanisms for including additional headers between the required header and the message.

The transition to IPv6 is proving to be gradual. To aid with this transition, some routers have been modified to interpret both IPv6 and IPv4 datagrams. For routers that cannot handle IPv6 addresses, IPv6-enabled routers can implement a technique called **tunneling,** which places IPv6 datagrams inside IPv4 datagrams. When other IPv6-enabled routers eventually receive the datagram, they can strip the IPv4 control information and continue as usual. This method is attractive because it allows routing to continue without interruption. However, it adds processing overhead at routers.[58, 59]

Self Review

1. How many possible IPv6 addresses are there?
2. What is the difference between anycast and multicast addresses?

Ans: **1)** There are 2^{128}, or approximately 3.4×10^{38}, unique IPv6 addresses. **2)** An anycast address allows a user to send a datagram to any one of a group of hosts. A multicast address allows a user to send a datagram to all hosts within a group.

16.8 Link Layer

The link layer interfaces with the **transmission medium** (often copper wiring or optical fiber). It is responsible for transforming a datagram (called a **frame** in the link layer) into a representation (such as electrical or optical) that is suitable for the specific transmission medium and for sending this representation into the medium. The link layer is also responsible for transforming that representation at the receiving computer back into bit streams that can be interpreted by the upper three layers of the TCP/IP stack. Because transmission media are physical entities, they are susceptible to interference which can cause errors. The link layer attempts to detect and, if possible, correct such errors.

Many implementations of the link layer use a **checksum** to determine if a frame was corrupted. A checksum is the result of a calculation on the bits in the frame. If the checksum calculated by the sender and inserted into the frame matches the checksum calculated by the receiver, it is likely that the frame was not corrupted during transmission. Some systems employ error-correcting codes that enable a receiver to both detect and correct corrupted frames. An example is a Hamming Code, which uses redundant bits to determine which bits in the transmission were corrupted and to correct the frames. If errors cannot be corrected, the sender must resend the original frame.

Self Review

1. What is the benefit of using an error-correcting code such as the Hamming Code?
2. What factors contribute to the overhead of using an error-correcting code such as the Hamming Code?

Ans: **1)** Error-correcting codes can be used to correct errors that occurred during transmission. This removes the need to retransmit corrupted frames. **2)** The complication with error-correcting codes is that they require additional bits to be sent with each frame. This can slow network transmission. These bits must be calculated at the sender and recalculated and checked at the receiver.

16.8.1 Ethernet

Ethernet is a type of LAN first developed at the Xerox Palo Alto Research Center in 1976 and later denned by the **IEEE 802.3** standard. Ethernet uses the **Carrier Sense Multiple** Access **with Collision Detection (CSMA/CD)** protocol. In 802.3-style CSMA/CD (i.e., CSMA/CD that conforms to the IEEE 802.3 standard), "intelligent" nodes are attached to the medium by hardware devices called **transceivers.** The nodes are deemed "intelligent" because a transceiver tests a shared medium to determine if it is available before transmitting data. When the station connected to the transceiver wishes to transmit, the transceiver sends data into the shared medium, while monitoring that medium to detect a simultaneous transmission called a **collision.** Due to delays in the medium, it is possible that multiple transceivers may decide that the medium is clear and begin transmitting simultaneously. When two frames collide, the data is corrupted.

If transceivers detect a collision, they continue to transmit bytes for a specific period of time to ensure that all transceivers become aware of the collision. Each transceiver, after learning of a collision, waits a random interval before attempting to transmit again. This interval is calculated to maximize throughput while minimizing new collisions—one of the most challenging issues in designing CSMA/CD technology. Ethernet employs a method called **exponential backoff.** The transceiver tracks how many times there has been a collision in trying to transmit a certain frame and uses it to calculate the delay before attempting to retransmit. This is called the **random delay.** Each time there is a collision, the length of the delay doubles—rapidly reducing the probability that the offending transceivers will retransmit at the same time. Eventually the frame is sent without collision.

While this method may seem inefficient, it actually works quite well in practice. If the delay were not random, then more collisions would be likely, resulting in greater loss of time. Due to the high transmission speeds and relatively small frames, multiple successive collisions after random retransmissions are usually rare and indicate a network configuration error or a hardware error. '

Self Review

1. What are the possible consequences if a transceiver does not continue to transmit bytes upon detection of a collision?

2. Why are collisions possible in Ethernet transmissions?

Ans: **1)** Other transceivers may not recognize that a collision has occurred. This can cause a transceiver to think a frame has been transferred correctly when it has in fact been corrupted. **2)** Transmission is not instantaneous, so two hosts can decide that the medium is free at the same time and start transmitting.

16.8.2 Token Ring

The **Token Ring** protocol operates on ring networks and employs **tokens** to gain access to the transmission medium. In a Token Ring, the token is actually an empty frame that circulates continuously between machines over a network having a logical ring topology (Fig. 16.3). When a node wishes to send a message, it must first wait for the empty frame to arrive. When the node receives the empty frame, it changes one bit in the header to indicate that the frame is no longer empty and writes its message and the address of the intended recipient to the frame. The node then sends the frame to its neighbor. Each node compares its own address to the address in the frame; if the addresses do not match, the node sends the frame on to the next node.

If the address does match, the machine copies the content of the message, changes the bit in the frame's header and passes the frame to the next node on the ring. When the original sender receives the frame (i.e., the frame has completed one cycle of the ring), it can determine if the message was received. At this point, the sender removes the message from the frame and passes the now-empty frame to its neighbor. The Token Ring protocol also contains complex mechanisms to protect the network from losing the token—if a station failed while holding the token, the token might never be released, thus deadlocking the network.

When the token is being passed without incident, the time to transmit a message is quite predictable, as is the time to recover from various errors. Token Rings are the most common network architecture after Ethernet.[64]

Self Review

1. What would happen if a station failed while it owned the token and there was no mechanism to recover?
2. Describe what happens when a host wants to send information using a Token Ring.

Ans: **1)** The token would never be released and all of the hosts would become deadlocked. **2)** The host must first wait until it receives the empty frame (the token). When the host receives the empty frame, it places its message in it. The frame is sent around the Token Ring until it reaches the destination host, which reads the message and indicates that the message has been read. The frame continues around the Token Ring until it reaches the first host, which then releases the token.

16.8.3 Fiber Distributed Data Interface (FDDI)

Fiber Distributed Data Interface (FDDI) shares many properties of the Token Ring protocol, but operates over fiber-optic cable, allowing it to support more transfers at greater speeds over larger distances. FDDI is built on two Token Rings,

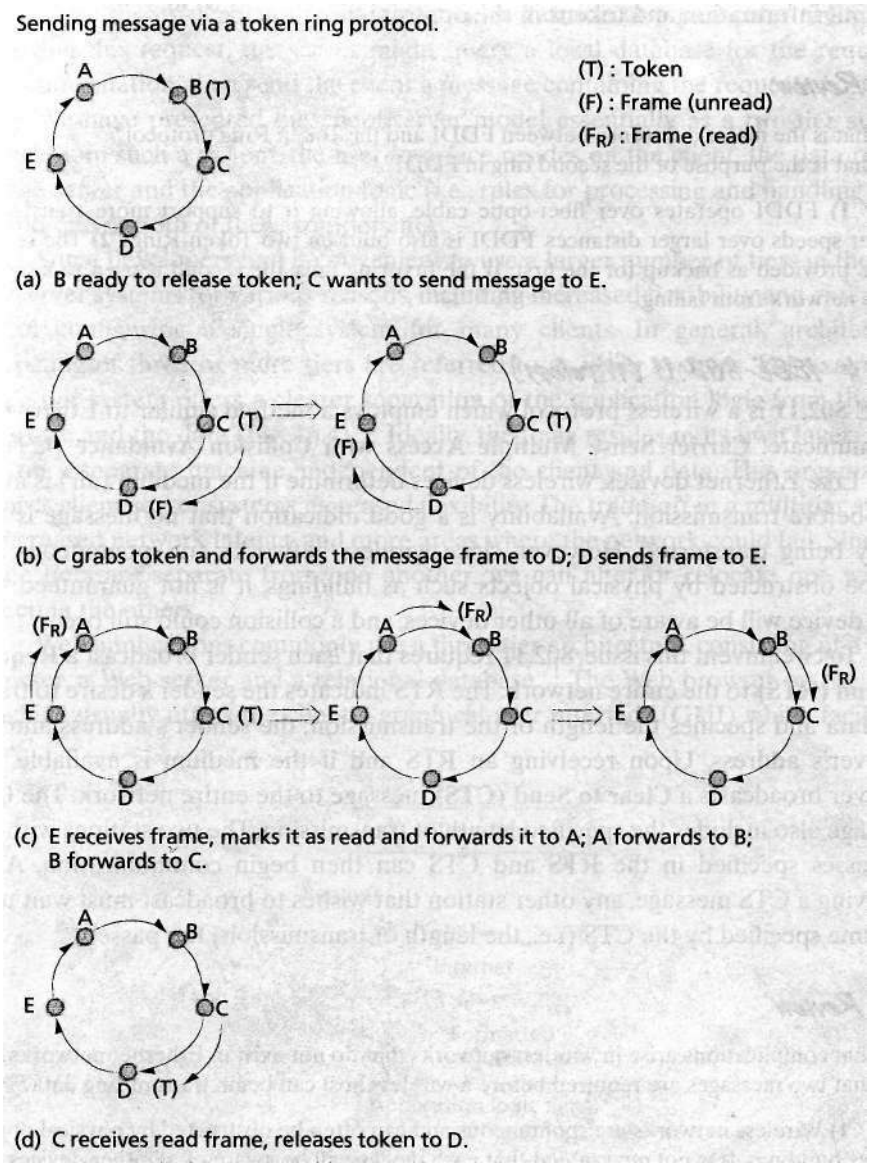


Figure 16.3 | *Sending a message via the Token Ring protocol.*

the second usually being reserved for backup. In FDDI, a token circulates around the optical fiber ring; stations cannot transmit until they obtain the token by receiving it from a preceding station. While the station is transmitting, no token is circulating, thus forcing all other stations to wait before they are allowed to transmit. When it has completed transmission, the transmitting station generates a new token, and other stations may attempt to capture it so they may transmit in turn. If

the secondary ring is not needed for backup purposes, FDDI will generally use it to transmit information and tokens in the opposite direction.[65, 66, 67]

Self Review

1. What is the major difference between FDDI and the Token Ring protocol?
2. What is the purpose of the second ring in FDDI?

Ans: **1)** FDDI operates over fiber-optic cable, allowing it to support more transfers at greater speeds over larger distances. FDDI is also built on two Token Rings. **2)** The second ring is provided as backup for the first. If the first ring fails, the second is used to keep the entire network from failing.

16.8.4 IEEE 802.11 (Wireless)

IEEE 802.11 is a wireless protocol which employs a method similar to Ethernet to communicate: **Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)**. Like Ethernet devices, wireless devices determine if the medium (air) is available before transmission. Availability is a good indication that no message is currently being transferred. However, since wireless networks are spontaneous and can be obstructed by physical objects such as buildings, it is not guaranteed that each device will be aware of all other devices, and a collision could still occur.

To circumvent this issue, 802.11 requires that each sender broadcast a **Request to Send (RTS)** to the entire network. The RTS indicates the sender's desire to transmit data and specifies the length of the transmission, the sender's address and the receiver's address. Upon receiving an RTS and if the medium is available, the receiver broadcasts a **Clear to Send (CTS)** message to the entire network. The CTS message also includes the specified length of transmission. The two stations with the addresses specified in the RTS and CTS can then begin communication. After receiving a CTS message, any other station that wishes to broadcast must wait until the time specified by the CTS (i.e., the length of transmission) has passed.[68]

Self Review

1. What complications arise in wireless networks that do not exist in Ethernet networks?
2. What two messages are required before a wireless host can begin transmitting data?

Ans: **1)** Wireless networks are spontaneous and can often be obstructed by physical objects such as buildings. It is not guaranteed that each device will be aware of all other devices, and collisions could still occur. **2)** The source host must send out an RTS and the destination host must respond with a CTS, before transmission is allowed to begin.

16.9 Client/Server Model

Many distributed applications operate according to a popular networking paradigm known as the **client/server model. Clients** are hosts that need various services performed, and servers are hosts that provide these services. Typically, clients transmit requests to servers over a network and servers process requests and return the

result to the client. For example, an Internet user might request from a server a list of airplane flights departing from a particular location at a particular time. Upon receiving this request, the server might query a local database for the requested flight information, then send the client a message containing the requested list.

We have presented the client/server model essentially as a two-tier system. Typically, in such a system, the user interface resides on the client, the data resides on the server and the application logic (i.e., rules for processing and handling data) lies on one or both of these components.

Some developers find it convenient to use a larger number of tiers in their client/server systems for various reasons, including increased flexibility and extensibility in configuring a single system for many clients. In general, architectures consisting of three or more tiers are referred to as n-tier systems. For example, a three-tier system offers a clearer separation of the application logic from the user interface and the data (Fig. 16.4).[70] Ideally, the logic resides in its own layer, possibly on a separate machine, independent of the client and data. This organization affords client/server systems increased flexibility. The trade-off in a multitier system is increased network latency and more areas where the network could fail. Since the three tiers are separate from one another, we can alter or relocate one without affecting the others.

Web applications commonly use a three-tier architecture, consisting of a client browser, a Web server and a relational database.[71] The Web browser on the client machine usually offers the client a graphical user interface (GUI) which facilitates

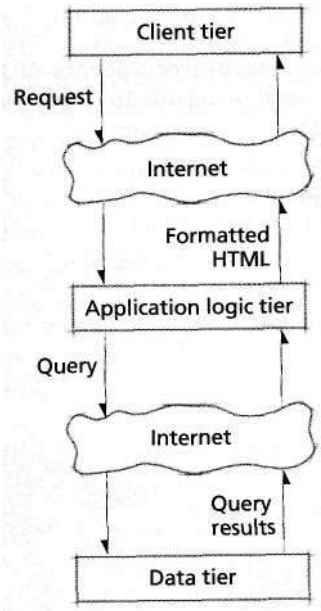


Figure 16.4 | *Three-tier client/server model.*

access to remote documents. The browser interprets pages written in HyperText Markup Language (HTML) and produces their representation on the client monitor. To retrieve a remote document, the browser communicates via HTTP with a Web server, which transmits HTML to the browser also via HTTP. A particular Web server might dynamically produce Web pages by querying a relational database (see Section 13.12.3). For example, imagine that a user visits an online auction site and wishes to view all the computers of a certain type that are currently being auctioned. A request would be sent from the client browser to the auction site's Web server. The server might then query a relational database residing on a remote machine and use the resulting information to dynamically create an HTML page. This page is sent to the browser, which interprets the HTML and displays the information on the client monitor. In this example, the client, server and data tiers are on physically separate machines.

Self Review

1. What are the names of the tiers in a typical three-tier system?
2. What are the benefits and complications associated with increasing the number of tiers of a Web-based system?

Ans: **1)** The client tier, the application logic tier and the data tier. **2)** The benefit is that it further modularizes the system. Each tier can be modified without altering the others. Adding tiers can introduce network latencies as requests are being fulfilled, diminishing application performance.

Web Resources

www.ietf.org/rfc
Provides an index of all of the Requests For Comments (RFCs) that have been published.

www.ietf.org
Internet Engineering Task Force provides links to its working groups and a discussion of the Internet standards process.

www.w3.org
World Wide Web Consortium (W3C) provides guidelines for many Web-based technologies.

www.its.bldrdoc.gov/fs-1037/
Federal Standards 1037 Web site provides a glossary of telecommunications terms.

www.iana.org/
Internet Assigned Numbers Authority sets default port numbers for many common protocols.

Summary

Network topology describes the relationship between different hosts, also called nodes, on a network. A logical topology displays which nodes in a network are directly connected.

Nodes on a bus network are connected to a single, common communication link. Since there are no intermediary nodes to retransmit the message, the length of bus communications medium must be limited to reduce attenuation.

Ring networks consist of a set of nodes, each maintaining exactly two connections to other nodes such that a message sent through one connection can eventually return via the other. Each node in the ring forwards each message, limiting attenuation but introducing a delay for retransmission.

In mesh networks at least two nodes have more than one path connecting them. A fully-connected mesh network directly connects every node to every other node. The pres-

ence of multiple paths between any two nodes increases the capacity available for network traffic, enabling higher network throughput.

Star networks contain a hub that is connected to all other nodes in the network. Star networks suffer from lower transmission delay than ring networks, because connections require only one intermediary node. If the central hub fails, however, messages cannot reach their recipients, so fault tolerance can be a problem.

Tree networks are hierarchical networks that consist of a root node and several subnodes, called children, that can have subnodes of their own. A tree topology can be used to join nodes that communicate with each other frequently into a subtree, thereby increasing network efficiency.

The proliferation of wireless network technology has introduced ad hoc networks. An ad hoc network is characterized as being spontaneous —any combination of wireless and wired devices may be connected to it at any time. The network topology is not fixed, which makes it difficult to have a network governed by a central node.

Networks can also be classified by the geographic dispersion of their hosts. A local area network (LAN) has limited geographic dispersion and is designed to optimize data transfer rates between its hosts. LANs interconnect resources using high-speed communication paths with optimized network protocols for local area environments. Advantages of local networks include error rates lower than those of larger networks, greater management flexibility, and independence from the constraints of the public networking system. Wide area networks (WANs) are broader, connecting two or more LANs; the largest WAN is the Internet. WANs generally employ a mesh topology, operate at lower speeds than LANs and have higher error rates, because they handle larger amounts of data being transmitted over far greater distances.

The TCP/IP protocol stack is composed of four logical levels called layers. The application layer is the highest level and provides protocols for applications to communicate. The transport layer is responsible for end-to-end com--unication. The network layer is responsible for moving the data on to the next step. The transport layer relies on the network layer to determine the proper path from one end of the communication to the other. The link layer provides an interface between the network layer and the underlying physical medium of the connection.

Application layer protocols specify the rules that govern remote interprocess communication and determine how processes should interact. Many of these protocols interact with resources on remote hosts. These resources are specified by a Uniform Resource Identifier (URI), which is a name that references a specific resource on the remote host.

The Hypertext Transfer Protocol (HTTP) is an application layer protocol that allows the transfer of a variety of data formats. HTTP defines a request for a resource and a response. The remote host processes the request and replies with a response, which contains in its header a code that tells the client whether the request was processed correctly or there was an error.

The File Transfer Protocol (FTP) is an application layer protocol that allows file-sharing between remote hosts. FTP specifies connections between two pairs of ports: one pair sends control information that governs the session, the other sends the actual data. After a connection is established, the client specifies actions for the FTP server to perform by issuing various requests to the server. The server attempts to satisfy each request, then issues a response specifying the result.

The transport layer is responsible for the end-to-end communication of messages. In a connection-oriented approach, hosts send each other control information—through a technique called handshaking—to set up a logical end-to-end connection. A connection-oriented approach imposes reliability on unreliable networks. By making communication reliable, these protocols guarantee that data sent from the sender will arrive at the intended receiver undamaged and in the correct sequence. In a connectionless approach, the two hosts do not handshake before transmission, and there is no guarantee that sent messages will be received in their original order, or at all.

The Transmission Control Protocol (TCP) is a connection-oriented transmission protocol. TCP guarantees that segments sent from a sender will arrive at the intended receiver undamaged and in the correct sequence. TCP handles error control, congestion control and retransmission, allowing protocols like HTTP and FTP to send information into the network as simply and reliably as writing to a file on the local computer.

When TCP is given a message to send over a network, TCP must first make a connection with the receiving host using a three-way handshake. The receiving host in TCP accounts for reordering of the segments by using the sequence numbers found in the message headers to reconstruct the original message. When the destination host gets one of these segments, it responds with an ACK segment containing the sequence number of the message. This guarantees that segments will eventually be received by the destination host and that they will be reassembled in the original order.

TCP also implements flow control and congestion control to regulate the amount of data sent by each host.

The connectionless User Datagram Protocol (UDP) provides the minimum overhead necessary for the transport layer. There is no guarantee that UDP datagrams will reach their destination in their original order, or at all.

The network layer receives segments from the transport layer and is responsible for sending these packets to the next stop toward the destination through a process known as routing. Routing is the two-step procedure of first determining the best route between two points and then sending packets along this route. Routers determine the next host for a given datagram based on information, such as network topologies and link quality, which includes strength of signal, error rate and interference which is broadcast throughout networks using various router protocols, such as the Routing Information Protocol (RIP).

Internet Protocol version 4 (IPv4) is the dominant protocol for directing information over a network. Destinations on the Internet are specified by IP addresses, which are 32-bit numbers in IPv4. To make networking simpler for the user, one or more names can be mapped to an IP address through the Domain Name System (DNS).

In the near future, there will be more addressable nodes on the Internet than available addresses using the current system. To combat this problem, the Internet Engineering Task Force introduced Internet Protocol version 6 (IPv6). IPv6 specifies three types of addresses: unicast, anycast and multicast. A unicast address describes a particular host on the Internet. Anycast addresses are designed to be sent to the nearest host with a particular address. Multicast addresses are designed to send packets to a group of hosts.

The link layer interfaces the software-oriented layer with the physical medium over which frames are sent. The link layer is responsible for detecting and, if possible, correcting transmission errors. Some systems employ error-correcting codes to correct the corrupted frames.

Ethernet uses the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol. In 802.3-style CSMA/CD, a transceiver tests a shared medium to determine if it is available before transmitting data. Due to delays in the medium, it is possible that multiple transceivers may decide that the medium is clear and begin transmitting simultaneously. If transceivers detect a collision caused by simultaneous transmissions, they continue to transmit bytes for a specific period of time to ensure that all transceivers become aware of the collision. Each transceiver, after learning of a collision, waits a random interval before attempting to transmit again.

The Token Ring protocol operates on ring networks and employs tokens to gain access to the transmission medium. In a Token Ring, a token that controls access to the transmission medium is actually an empty frame that is continuously circulated between machines over a network having a logical ring topology. When a machine owns the token, it generates data, places it in the frame and sends the frame to its neighbor. Each machine forwards the token until it reaches its destination. At the destination, the machine copies the content of the message, marks the frame as having been delivered and passes the frame to its neighbor. When the original sender receives the frame, it removes the message from the frame and passes the token to its neighbor.

Fiber Distributed Data Interface (FDDI) operates over fiber-optic cable, allowing it to support more transfers at greater speeds over larger distances. FDDI is built on two Token Rings, the second usually being reserved for backup.

802.11 wireless communications employ a method similar to Ethernet to communicate: Carrier Sense Multipie Access with Collision Avoidance (CSMA/CA). This requires that each sender broadcast a Request to Send (RTS) to the entire network. Upon receiving an RTS, the receiver broadcasts a Clear to Send (CTS) message to the entire network if the medium is available.

Typically, in a two-tier client/server system, the user interface resides on the client, the data resides on the server and the application logic lies on one or both of these components. In place of this two-tier model, client/server systems often employ a three-tier system, which offers a clearer separation of the application logic from the user interface and the data. Ideally, the logic resides in its own layer, possibly on a separate machine, independent of the client and data. This organization affords the client/server system increased flexibility and extensibility. The trade-off in a multitier system is increased network latency and more areas where the network could fail.

Key Terms

acknowledgement segment (ACK)—In TCP, a segment that is sent to the source host to indicate that the destination host has received a segment. If a source host does not receive and ACK for a segment, it will retransmit that segment. This guarantees that each transmitted segment is received.

ad hoc network—Network characterized as being spontaneous—any number of wireless or wired devices may be connected to it at any time.

any cast—Type of IPv6 address that enables a datagram to be sent to any host within a group of hosts.

application layer (in OSI)—Interacts with the applications and provides various network services, such as file transfer and e-mail.

application layer (in TCP/IP)—Protocols in this layer allow applications on remote hosts to communicate with each other. The application layer in TCP/IP performs the functionality of the top three layers of OSI—the application, presentation and session layers.

attenuation—Deterioration of a signal due to physical characteristics of the medium.

bus network—Network in which the nodes are connected by a single bus link (also known as a linear network).

Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)—Protocol used in 802.11 wireless communication. Devices must send a Request To Send (RTS) and receive a Clear To Send (CTS) from the destination host before transmitting.

Carrier Sense Multiple Access with Collision Detection (CSMA/CD)-Protocol used in Ethernet that enables transceivers to test a shared medium to see if it is available before transmitting data. If a collision is detected, transceivers continue transmitting data for a period of time to ensure that all transceivers recognize the collision.

checksum—Result of a calculation on the bits of a message. The checksum calculated at the receiver is compared to the checksum calculated by the sender (which is embedded in the control information). If the checksums do not match, the message has been corrupted.

Clear to Send (CTS)—Message that a receiver broadcasts in the CSMA/CA protocol to indicate that the medium is free. A CTS message is sent in response to a Request to Send (RTS).

client/server model—Popular networking paradigm in which processes that need various services performed (clients) transmit their requests to processes that provide these services (servers). The server processes the request and returns the result to the client. The client and the server are typically on different machines on the network.

client—Process in the client/server model that needs various services performed.

collision—Simultaneous transmission in CSMA/CD protocol.

congestion control—Means by which TCP restricts the number of segments sent by a single host in response to network congestion.

connectionless transport—Method of implementing the transport layer in which there is no guarantee that data will arrive in order or at all.

connection-oriented transport—Method of implementing the transport layer in which hosts send control information to govern the session. Handshaking is used to set up the connection. The connection guarantees that all data will arrive and in the correct order.

control information—Data in the form of headers and/or trailers that allows protocols of the same layer on different machines to communicate. Control information might include the addresses of the source and destination hosts and the type of data or size of the data that is being sent.

data link layer (in OSI)—At the sender, converts the data representation from the network layer into bits to be transmitted over the physical layer. At the receiver, converts the bits into the data representation for the network layer.

datagram—Piece of data transferred using UDP or **IP.**

Domain Name System (DNS)—System on the Internet used to translate a machine's name to an IP address.

Ethernet (IEEE 802.3)—Network that supports many speeds over a variety of cables. Ethernet uses the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol. Ethernet is the most popular type of LAN.

exponential backoff—Method employed by Ethernet to calculate the interval before retransmission after a collision; this reduces the chance of subsequent collisions on the same transmission, thus increasing throughput.

Fiber Distributed Data Interface (FDDI)—Protocol that shares many properties of a Token Ring, but operates over fiber-optic cable, allowing the transfer of more information at greater speeds. In FDDI, a token circulates around the optical fiber ring; stations cannot transmit until they obtain the token by receiving it from a preceding station. FDDI uses a second Token Ring as backup or to circulate tokens in the reverse direction of the primary Token Ring.

File Transfer Protocol (FTP)—Application layer protocol that moves files between different hosts on a network. FTP specifies connections between two pairs of ports: one pair sends control information that governs the session, the other sends the actual data.

flow control—Means by which TCP regulates the number of segments sent by a host to avoid overwhelming the receiver.

frame—Piece of data in the link layer. Contains both the message and the control information.

fully-connected mesh network—Mesh network in which each node is directly connected to every other node. These networks are faster and more fault tolerant than other net-

works, but also unrealizable on all but the smallest of networks because of the cost of the potentially enormous number of connections.

handshaking—Mechanism in a connection-oriented transport layer in which hosts send control information to create a logical connection between the hosts.

header—Control information placed in front of a data message.

host—Entity, such as a computer or Internet-enabled cellular phone, that receives and/or provides services over a network. Also called a node.

hub—Central node (such as in a star network) responsible for relaying messages between nodes.

HTTP request—Resource request from an HTTP client to an HTTP server.

HTTP response—Reply message from an HTTP server to an HTTP client, consisting of a status, header and data.

Hypertext Transfer Protocol (HTTP)—Application layer protocol used for transferring HTML documents and other data formats between a client and a server.

IEEE 802.11—One of the standards that governs wireless communication. It dictates that hosts follow the CSMA/CA protocol.

Internet Protocol version 6 (IPv6)—New version of the Internet Protocol that uses 128-bit addresses and specifies three types of addresses: unicast, anycast and multicast.

Internet Protocol (IP)—Primary protocol for directing information over a network. Destinations on the Internet are specified by 32-bit numbers called IP addresses.

IP address—Address of a particular host on the Internet.

layer—Level of abstraction in the TCP/IP protocol stack associated with certain conceptual functions. These layers are the application layer, transport layer, network layer and link layer.

link—Medium over which services are physically transmitted in a network.

link layer (in TCP/IP) —Responsible for interfacing with and controlling the physical medium over which data is sent.

local area network (LAN)—Type of network used to interconnect resources using high-speed communication paths optimized for local area environments, such as office buildings or college campuses.

logical topology—Map of a network that depicts which nodes are directly connected.

mesh network—Network in which at least two nodes have more than one path connecting them. Faster and more fault tolerant than all but fully-connected mesh networks.

multicast—Type of IPv6 address used to send packets to all hosts in a group of related hosts.

Multipurpose Internet Mail Extensions (MIME)—Electronic mail standard defining five content types: text, image, audio, video and application.

network layer—Protocols responsible for sending data to the next host toward the destination. This layer exists in both the TCP/IP model and the OSI model of network communication.

network topology—Representation of the relationships of nodes in a network. Some examples are bus networks ring networks, star networks, tree networks and mesh networks.

n-tier system—Architecture for network-based applications The three-tier system, for example, has a client tier, an application logic tier and a data tier.

physical layer (in OSI)—Transmits bits over physical media, such as cables. The data link layer and physical layer in OSI correspond to the link layer in TCP/IP.

port—Identifies the specific socket on a machine to which to send data. For example, HTTP communicates by default on port 80.

presentation layer (in OSI)—Solves compatibility problems by translating the application data into a standard format that can be understood by other layers.

protocol—Set of rules that govern how two entities should interact. Common examples include Transmission Control Protocol (TCP), Internet Protocol (IP) and Hypertext Transfer Protocol (HTTP).

random delay—Interval of time calculated by the exponential backoff method of CSMA/CD before a transceiver can retransmit a frame after a collision.

reliable network—Network which does not damage or lose packets.

Request to Send (RTS)—Message sent from a wireless device in the CSMA/CA protocol that indicates a desire to transmit data, the length of the transmission, the sender address and the receiver address. If the medium is available, the receiver will send a Clear To Send (CTS) message.

ring network—Network consisting of a set of nodes, each maintaining exactly two connections to other nodes in that network. These networks have a low fault tolerance since the failure of any single node can cause the whole network to fail.

router—Computer that is an intermediate destination between the sending host and the receiving host. The router is responsible for determining where to send a datagram next in order for it to eventually reach its destination.

Routing Information Protocol (RIP)—Protocol that defines how routing information is propagated throughout networks. RIP requires routers to share their entire routing table with other routers; this limits its use to small networks.

routing table —Representation of a network used to determine where routers should send datagrams next on their path to their destination.

routing—Determining the best route between two points and sending packets along this route.

segment—Piece of data sent by TCP. It includes the message and the TCP header.

server—Process in the client/server model that performs various services for clients.

session layer (in OSI)—Establishes, manages and terminates the communication between two end users.

socket - Software construct that represents one endpoint of a connection.

star network—Network containing a hub that is directly connected to all other nodes in the network. The hub is responsible for relaying messages between nodes.

Symbian OS —Small operating system for smart phones (mobile phones with the functionality of a PDA).

synchronization segment (SYN)—In TCP, the first handshaking segment sent; contains the sequence number of the source host.

synchronization/acknowledgement segment (SYN/ACK)—In TCP, the second handshaking segment sent; acknowledges that the SYN segment was received and contains the sequence number of the destination host.

TCP/IP Protocol Stack—Hierarchical decomposition of computer communications functions into four levels of abstraction called layers. These layers are the application layer, transport layer, network layer and link layer.

TCP window—Flow-control and congestion-control mechanism in which only a certain amount of data can be sent by the network layer without the receiver explicitly authorizing the sender to send more.

three-tier system—System which offers a separation of the application logic, the user interface and the data. The user interface tier (also called the client tier) communicates with the user. The application logic tier is responsible for the logic associated with the system's function. The data tier stores the information that the user wishes to access.

token—Empty frame used to ensure that only one host is transmitting data at a time in the Token Ring and FDDI protocols.

Token Ring—Protocol in which a token is circulated around a ring network. Only one host can own the token at a time, and only its owner can transmit data.

trailer—Control information appended to the end of a data message.

transceiver—Hardware device that attaches an Ethernet node to the network transmission medium. Transceivers test a shared medium to see if it is available before transmitting data, and monitor the medium to detect a simultaneous transmission called a collision.

Transmission Control Protocol (TCP) —Connection-oriented transmission protocol designed to provide reliable communication over unreliable networks.

transmission medium—Material used to propagate a signal (e.g., optical fiber or copper wire).

transport layer—Set of protocols responsible for end-to-end communication of data in a network. This layer exists in both the TCP/IP model and the OSI model of network communication.

tree network—Hierarchical network that consists of multiple star networks. The hub of the first star network is the root of the tree. Each node that this hub connects serves as a hub for another star network and is a root of a subtree.

tunneling—Process of placing IPv6 datagrams in the body of IPv4 datagrams when communicating with routers that do not support IPv6.

two-tier system—A system in which the user interface resides on the client, the data resides on the server and the application logic lies on one or both of these components.

unicast address—IP address used to deliver data to a single host.

Uniform Resource Identifier (URI)—Name that references a specific resource on the Internet.

Uniform Resource Locator (URL) A URI used to access a resource in a common protocol such as HTTP and FTP. Consists of the protocol, host name, port and path of the resource.

unreliable network—Network that may damage or lose packets.

User Datagram Protocol (UDP) —Connectionless transmission protocol which allows datagrams to arrive out of order, duplicated or not at all.

wide area network (WAN)—Type of network connecting two or more local area networks, usually operating over great geographical distances. WANs are generally implemented with a mesh topology and high-capacity connections. The largest WAN is the Internet.

Exercises

16.1 What are the pros and cons of using a layered network architecture?

16.2 Why does TCP/IP not specify a single protocol at each layer?

16.3 Compare and contrast connection-oriented services and connectionless services.

16.4 Distinguish between addressing, routing, and flow control. What is a windowing mechanism?

16.5 Explain the operation of CSMA/CD. Compare the predictability of CSMA/CD with token-based approaches. Which of these schemes has the greatest potential for indefinite postponement?

16.6 One interesting problem that occurs in distributed control, token-passing systems, is that the token may get lost. Indeed, if a token is not properly circulating around the network, no station can transmit. Comment on this problem. What safeguards can be built into a token-passing network to determine if a token has been lost, and then to restore proper operation of the network?

16.7 The text said that the Internet is implemented as a mesh network. What would be the consequences if the internet were implemented as a tree network? If it was implemented as a ring network? If it was implemented as a fully-connected mesh network?

16.8 HTTP and FTP are both implemented on top of TCP What would be the effect if they were implemented on top of UDP?

16.9 Why was CSMA/CD not used for wireless systems?

16.10 Why would an implementation of the link layer utilize error-correcting codes? How would that affect the system?

Suggested Projects

16.11 A number of protocols have been proposed to replace TCP. Researchers at MIT and Berkeley have recently developed the eXplicit Control Protocol (XCP) which is designed to speed up transmissions. Research this protocol and examine what changes it makes to TCP.

16.12 In the text, we mentioned that Routing Information Protocol (RIP) is a means to determine the next host for a datagram. Research RIP and explore how it determines the next host for a datagram.

Suggested Simulations

16.13 Write a simulation program to experiment with routing procedures. Create a mesh network containing some number of nodes and links. Create your own routing protocol to send information about the links throughout the network. Check to see that each node can efficiently determine the next node along a path between two nodes in the graph. Experiment with "crashing" a node or changing the network topology. How quickly will all of your nodes figure out the new topology?

Recommended Reading

Many books go into greater depth on networking than space permits in an operating systems text. Those include Tanenbaum's classic *Computer Networks,* 4th ed. and *Computer Networking: A Top-Down Approach Including the Internet* by Kurose and Ross.[73, 74]

Requests for Comments (RFCs) describe networking protocols, including TCP, IP, UDP, FTP and SMTP. The Internet Engineering Task Force (IETF) provides free access to every RFC via their Web site (www.ietf.org).

Networking remains a rich research area. Papers such as "Congestion Control for High Bandwidth-Delay Product Networks" and "Why Wi-Fi Wants to be Free" give a technical and more general idea of how this exciting research area is progressing.[75, 76] Professional groups such as the Institute of Electrical and Electronics Engineers (IEEE; www.ieee.org) and the Association for Computing Machinery (ACM; www.acm.org) are an ideal source of new literature on the topic. Interesting papers available through the ACM include "Frameworks for Component-Based Client/Server Computing," "The Transport Layer: Tutorial and Survey" and "Fundamental Challenges in Mobile Computing."[77, 78, 79] The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e Bibliography.pdf.

Works Cited

1. *Federal Standards On-Line,* "Network Topology," <www.its.bldrdoc.gov/fs-1037/>.
2. Abeysundara, B. W., and A. E. Kamal, "High-Speed Local Area Networks and Their Performance: A Survey," *ACM Computing* Surveys, Vol. 23, No. 2, June 1991, p. 231.
3. *Federal Standards On-Line,* "Network Topology," <www.its.bldrdoc.gov/fs-1037/>.
4. Abeysundara, B. W., and A. E. Kamal, "High-Speed Local Area Networks and Their Performance: A Survey," *ACM Computing Surveys,* Vol. 23, No. 2, June 1991, pp. 248-249.
5. *Federal Standards On-Line,* "Network Topology," <www.its.bldrdoc.gov/fs-1037/>.
6. Abeysundara, B. W., and A. E. Kamal, "High-Speed Local Area Networks and Their Performance: A Survey," *ACM Computing* Surveys, Vol. 23, No. 2, June 1991, pp. 248-250.
7. *Federal Standards On-Line,* "Network Topology," <www.its.bldrdoc.gov/fs-1037/>.
8. Abeysundara, B. W., and A. E. Kamal, "High-Speed Local Area Networks and Their Performance: A Survey," *ACM Computing Surveys,* Vol. 23, No. 2, June 1991, pp. 248-250.
9. *Federal Standards On-Line,* "Network Topology," <www.its.bldrdoc.gov/fs-1037/>.
10. Rajaraman, R., "Topology Control and Routing in Ad Hoc Networks: A Survey," *ACM SIGACT News,* Vol. 33, No. 2, June 2002, pp. 60-74.
11. Symbian, "Symbian: About Us," <www.symbian.com/about/about.html>.
12. Blackwood, X, "Why You Can't Ignore Symbian," May 22, 2002, <techupdate.zdnet.com/techupdate/stories/main/0,14179,2866892,00.html>.
13. Mery, D., "Symbian Technology: Why Is a Different Operating System Needed?" March 2002, <www.symbian.com/technology/why-diff-os.html>.
14. Blackwood, X, "Why You Can't Ignore Symbian," May 22, 2002, <techupdate.zdnet.com/techupdate/stories/main/0,14179,2866892,00.html>.
15. Mery, D., "Symbian Technology: Why Is a Different Operating System Needed?" March 2002, <www.symbian.com/technology/why-diff-os.html>.
16. Mery, D., "Symbian Technology: Symbian OS v7.0 Functional Description," March 2003, <www.symbian.com/technology/symbos-v7x-det.html>.
17. Mery, D., "Symbian Technology: Why Is a Different Operating System Needed?" March 2002, <www.symbian.com/technology/why-diff-os.html>.
18. PASC, "The Portable Application Standards Committee," <www.pasc.org>.
19. Mery, D., "Symbian Technology: Why Is a Different Operating System Needed?" March 2002, <www.symbian.com/technology/why-diff-os.html>.
20. Mery, D., "Symbian Technology: Symbian OS v7.0 Functional Description," March 2003, <www.symbian.com/technology/symbos-v7x-det.html>.
21. de Jode, M., "Symbian: Technology: Standards: Symbian on Java," June 2003, <www.symbian.com/technology/standard-java.html>.
22. Dixon, K., "Symbian OS Version 7.0s: Functional Description," Rev. 2.1, June 2003, <www.symbian.com/technology/symbos-v7s-det.html>.
23. Dixon, K., "Symbian OS Version 7.0s: Functional Description," Rev. 2.1, June 2003, <www.symbian.com/technology/symbos-v7s-det.html>.
24. "WCDMA," *SearchNetworking.com,* July 2003, <searchnetworking.techtarget.com/sDefinition/0,,sid7_gci505610,00.html>.
25. Abeysundra, B. W., and A. E. Kamal, "High-Speed Local Area Networks and Their Performance: A Survey," *ACM Computing Surveys,* Vol. 23, No. 2, June 1991, pp. 223-250.
26. *Federal Standards On-Line,* "Network Topology," <www.its.bldrdoc.gov/fs-1037/>.
27. Yanowitz, Jason, "Under the Hood of the Internet: An Overview of the TCP/IP Protocol Suite," <www.acm.org/crossroads/xrdsl-1/tcpjmy.html>.
28. TCP/IP and OSI, <searchnetworking.techtarget.com/originalContent/0,289142,sid7_gci851291,00.html>.
29. Tyson, J., "How OSI Works—The Layers," <computer.howstuffworks.com/osi1.htm>.
30. Tyson, X, "How OSI Works—Protocol Stacks," <computer. howstuffworks.com/osi2.htm>.
31. "The 7 Layers of the OSI Model," <webopedia.internet.com/quick_ref/OSI_Layers.asp>.
32. Module 4: Standards and the ISO OSI Model, "ISO OSI-Application Layer (7)," <vvv.it.kth.se/edu/gru/Telesys/96P3_Telesystem/HTML/Module4/ISO-15.html>, modified 11/1/95, © 1995 G. Q. Maguire Jr., KTH/Teleinformatics.
33. "Naming and Addressing: URIs, URLs.," *W3C* July 2002, <http://www.w3.org/Addressing/>.
34. "Port," *searchNetworking.com,* last modified July 2003, <searchnetworking.techtarget.com/sDefinition/0,,sid7_gci212807,00.html>.
35. "Hypertext Transfer Protocol-HTTP/1.1," RFC 2616, June 1999, <www.ietf.org/rfc/rfc2616.txt>.
36. "Multipurpose Internet Mail Extensions (MIME), Part One: Format of Internet Message Bodies," RFC 2045, November 1996, <www.ietf.org/rfc/rfc2045.txt>.
37. "Uniform Resource Identifiers (URI): General Syntax," RFC 2396, August 1998, <www.ietf.org/rfc/rfc2396.txt>.
38. "File Transfer Protocol (FTP)," RFC 959, October 1985, <www.ietf.org/rfc/rfc0959.txt>.

39- Pouzin, L., "Methods, Tools, and Observations on Flow Control in Packet-Switched Data Networks," *IEEE Transactions on Communications,* April 1981.

40. George, F. D., and G. E. Young, "SNA Flow Control: Architecture and Implementation," *IBM Systems Journal,* Vol. 21, No. 2, 1982, pp. 179-210.

41. Grover, G. A., and K. Bharath-Kumar, "Windows in the Sky-Flow Control in SNA Networks with Satellite Links," *IBM Systems Journal,Vo\.* 22, No. 4.1983, pp. 451-463.

42. Henshall, J., and S. Shaw, *OSI Explained: End-to-End Computer Communication Standards,* Chichester, England: Ellis Horwood Limited, 1988.

43. Iren, S.; P. Amer; and P. Conrad, "The Transport Layer: Tutorial and Survey," *ACM Computing Surveys,* Vol. 31, No. 4, December 1999.

44. "Explanation of the Three-Way Handshake via TCP/IP," <support.microsoft.com:80/support/kb/articles/Q172/9/83.ASP>.

45. "Transmission Control Protocol, DARPA Internet Program, Protocol Specification," RFC 793, September 1981, <www.ietf.org/rfc/rfc0793.txt>.

46. Iren, S.; P. Amer; and P. Conrad, "The Transport Layer: Tutorial and Survey," *A CM Computing Surveys,* Vol. 31, No. 4, December 1999, pp. 360-405.

47. *Federal Standards On-Line,* "Network Topology," <www.its.bldrdoc.gov/fs-1037/>.

48. "The TCP Maximum Segment Size and Related Topics," RFC 789,November 1983, <www.faqs.org/rfcs/rfc879.html>.

49. Deitel, H.; P. Deitel; and S. Santry, *Advanced Java 2 Platform: How to Program,* Prentice Hall, Upper Saddle River, NJ, 2002, pp. 530-592.

50. "User Datagram Protocol," RFC 768, August 1980, <www.ietf.org/rfc/rfc0768.txt>.

51. Iren, S.; P. Amer; and P. Conrad, "The Transport Layer: Tutorial and Survey," *ACM Computing Surveys,* Vol. 31, No. 4, December 1999, pp. 360-405.

52. "Overview of the TCP/IP Protocols," *IT Standards Handbook—The Internet Protocol Suite (Including TCP/IP).* <www.nhsia.nhs.uk/napps/step/pages/ithandbook/h322-4.htm>.

53. "How Domain Name Servers Work," <computer.howstuffworks.com/dns.htm>.

54. "DNS Demystified," <www.lantimes.com/handson/97/706al07a.html>.

55. "Exploring the Domain Name Space," <hotwired.lycos.com/webmonkey/webmonkey/geektalk/97/03/index4a.html>.

56. "Experimental Internet Stream Protocol, Version 2 (ST-II)," RFC 1190, October 1990. <www.ietf.org/rfc/rfcl190.txt>.

57. "Internet Protocol Version 6 (IPv6) Addressing Architecture," RFC 3513, April 2003, <www.ietf.org/rfc/rfc3513.txt>.

58. "Internet Protocol, Version 6 (IPv6) Specification," RFC 1883, December 1993, <www.ietf.org/rfc/rfcl883.txt>.

59. "IP Version 6 Addressing Architecture," RFC 1884, December 1995, <www.ietf.org/rfc/rfcl884.txt>.

60. "Cyclic Redundancy Check," *Free On-Line Dictionary of Computing,* last modified August 2003, <http://wombat.doc.ic.ac.uk/foldoc/foldoc.cgi?CRC>.

61. Duerincks, G., "Cyclic Redundancy Checks in Ada95," *ACM SIGAda Ada Letters,* Vol. 17, No. 1, January/February 1997. pp. 41-53.

62. Chae, L., "Fast Ethernet: 100BaseT," *Network Magazine,* December 1995, <www.networkmagazine.com/article/NMG20000727S0014>.

63. Intel, "Gigabit Ethernet Solutions (White Paper)," 2001, <www.intel.com/network/connectivity/resources/doc_library/white_papers/gigabit_ethernet/gigabit_ethernet.pdf>.

64. "Token Ring," *searchNetworking.com,* last modified July 2003, <searchnetworking.techtarget.com/sDefinition/0, ,sid7_gci213154,00.html>.

65. Mokhoff, N., "Communications: Fiber Optics," *IEEE Spectrum,* January 1981.

66. Schwartz, M., "Optical Fiber Transmission —from Conception to Prominence in 20 Years," *IEEE Communications Magazine.* May 1984.

67. "FDDI," *searchNetworking.com,* last modified April 2003 <searchnetworking.techtarget.com/gDefinition/0,294236,sid7_gci213957,00.html>.

68. Karve, A., "802.11 and Spread Spectrum," *Network Magszine,* December 1997, <www.networkmagazine.com/article/NMC20000726S0001>.

69. Sinha, A., "Client-Server Computing," *Communications of the ACM,* Vol. 35, Issue 7, July 1992, pp. 77-98.

70. Lewandowski, S., "Frameworks for Component-Based Client Server Computing." *ACM Computing Surveys,* Vol. 30, No. 1, March 1998. pp. 3-27.

71. Lewandowski, S., "Frameworks for Component-Based Client Server Computing," *ACM Computing Surveys,* Vol. 30, No. 1, March 1998, pp. 3-27.

72. Katabi, D.; M. Handley; and C. Rohrs, "Congestion Control for High Bandwidth-Delay Product Networks," *Proceedings of the 2002 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications,* August 2002, pp. 89-102.

73. Tanenbaum, A., *Computer Networks,* 4th ed., Upper Saddle River, NJ, Prentice Hall PTR, 2002.

74. Kurose, J., and K. Ross, *Computer Networking: A Top-Down Approach Featuring the Internet,* Reading, MA. Addison-Wesley 1st ed., 2000.

75. Katabi, D.; M. Handley; and C. Rohrs, "Congestion Control for High Bandwidth-Delay Product Networks," *Proceedings of the 2002 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications,* August 2002, pp. 89-102.

76. Schmidt T., and A. Townsend, "Why Wi-Fi Wants to Be Free," *Communications of the ACM*, Vol. 46, No. 5, May 2003, pp. 47-52.
77. Lewandowski, S., "Frameworks for Component-Based Client/ Server Computing," *ACM Computing Surveys,* Vol. 30, No. 1, March 1998, pp. 3-27.
78. Iren, S.; P. Amer; and P. Conrad, "The Transport Layer: Tutorial and Survey," *ACM Computing Surveys, Vol.* 31, No. 4, December 1999, pp. 360-405.
79. Satyanarayanan, M., "Fundamental Challenges in Mobile Computing," *Symposium on Principles of Distributed Computing '96,* ACM, 1996, Philadelphia PA.

I wish to have no connection with any ship that does not sail fast;for I intend to go in harm's way.

—John Paul Jones—

Whatever shall we do in that remote spot?

—Napoleon Bonaparte—

The art of progress is to preserve order amid change and to preserve change amid order.

-Alfred North Whitehead—

Order and simplification are the first sted toward the mastery of a subject - the actual enemy is the unknown.

—Thomas Mann—

The clock, not the steam-engine, is the key machine of the modern industrial age.

—Lewis Mumford—

It's so awkward to tell one client that you're working on someone else's business...

—Andrew Forthingham—

You may delay, but Time will not.

—Benjamin Franklin—

Chapter 17

Introduction to Distributed Systems

Objectives

After reading this chapter, you should understand:

- *the need for distributed computing.*
- *fundamental properties and desirable characteristics of distributed systems.*
- *remote communication in distributed systems.*
- *synchronization, mutual exclusion and deadlock in distributed systems.*
- *examples of distributed operating systems.*

Chapter Outline

17.1 *Introduction*

17.2 *Attributes of Distributed* ***Systems***

17.2.1 Performance and Scalability

17.2.2 Connectivity and Security

17.2.3 Reliability and Fault Tolerance

17.2.4 Transparency

17.2.5 Network Operating Systems

17.2.6 Distributed Operating Systems

17.3 *Communication in Distributed* ***Systems***

17.3.1 Middleware

Anecdote: *Consequences of Errors*

17.3.2 Remote Procedure Call (RPC)

17.3.3 Remote Method Invocation (RMI)

17.3.4 CORBA (Common Object Request Broken Architecture)

17.3.5 DCOM (Distributed Component Object Model)

17.3.6 Process Migration in Distributed Systems

17.4 ***Synchronization in Distributed*** *Systems*

17.5 ***Mutual Exclusion in Distributed*** *Systems*

17.5.1 Mutual Exclusion withous Shared Memory

17.5.2 Agrawala and Ricart's Distributed Mutual Exclusion Algorithm

17.6 ***Deadlock in Distributed Systems***

17.6.1 Distributed Deadlock

17.6.2 Deadlock Prevention

17.6.3 Deadlock Detection

17.6.4 A Distributed Resource Deadlock Algorithm

17.7 ***Case Study: The Sprite Distributed Operating System***

17.8 *Case Study: The Amoeba* ***Distributed Operating System***

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

17.1 Introduction

As the speed and reliability of networking have improved, computers worldwide have become increasingly interconnected. Remote communication via networks, originally reserved for large computer installations and academic environments, has become pervasive. In **distributed systems,** remote computers cooperate via a network to appear as a local machine. Both distributed systems and network systems spread computation and storage throughout a network of computers. However, users of a **distributed operating system** are given the impression that they are interacting with just one machine, whereas users of a network operating system must be aware of the distributed implementation. Applications of distributed systems are able to execute code on local machines and remote machines and to share data, files and other resources among these machines.

Distributed systems often arise from the need to improve the capacity (e.g., processing power and storage size) and reliability of a single machine. Economic factors can limit the capabilities of a system. By implementing an operating system across several inexpensive machines, it is possible to design a powerful system without expensive machines. For example, it is difficult to connect hundreds of processors on a single mainboard; moreover, most hard disk interfaces do not support the hundreds of disks required for terabytes of storage. Having a single machine with an exorbitant amount of resources is wasteful; a single user would rarely, if ever, take advantage of such capacity. Dividing the resources among a group of machines lets multiple users share the resources, while guaranteeing that there will be enough for the occasional large job. Another reason to adopt a distributed system is to serve a large user base. A **distributed file system** places files on separate machines, while providing the view of a single file system. Distributed file systems can allow vast numbers of users to access the same set of files reliably and efficiently.

Although distributed systems offer many advantages over single-machine operating systems, they can be complex and difficult to implement and manage. For example, they must handle communication delays and reliability problems introduced by the underlying networks. It is harder to manage machine failure in distributed systems; an operating system distributed across n machines is far more likely to experience a system crash than a single-machine operating system. Another challenge is ensuring that each computer in the system has the same view of the entire system.

The discussion of distributed systems in this and the next chapter builds on Chapter 16. In this chapter we discuss distributed communication, distributed file systems and distributed processing. Distributed communication describes how processes on different machines communicate, while distributed processing describes the interaction among distributed processes. Distributed file systems provide a means of managing files within a distributed system.

Self Review

1. What are some benefits of a distributed system?
2. What makes distributed systems complex and difficult to implement and manage?

Ans: **1)** Distributed systems can achieve a high level of performance for a lesser cost than with a single system. Distributed systems can also allow vast numbers of users access to the same files reliably and efficiently. **2)** Handling communication delays and reliability problems-introduced by the underlying networks, responding to machine failure and ensuring that each computer in the system has the same view of the entire system.

17.2 Attributes of Distributed Systems

For decades, researchers and professionals have stressed the importance of distributed systems. The explosion of the Internet that occurred with the popularization of the World Wide Web in 1993 has made distributed systems common. This section discusses performance, scalability, connectivity, security, reliability and fault tolerance in distributed systems.

17.2.1 Performance and Scalability

In a centralized system, a single server handles all user requests. With a distributed system, user requests can be sent to different servers working in parallel to increase performance. **Scalability** allows a distributed system to grow (i.e., add more machines to the system) without affecting the existing applications and users.

Self Review

1. How can a distributed systems increase performance?
2. How does scalability make distributed systems better than centralized systems?

Ans: **1)** Multiple servers can handle user requests simultaneously. **2)** Scalability allows tin-distributed system to grow (i.e., add more machines) without affecting the existing applications and users.

17.2.2 Connectivity and Security

A distributed system can provide seamless access to resources distributed across the network. If the resource is a processor, the distributed system should allow tasks to be executed on any machine. If the resource is a globally shared file system, then remote users should be able to access the file system as they would access a local, private file system. For example, a user should be able to navigate the file system and open a file using a graphical browser or shell instead of connecting to an FTP program.

Connectivity in distributed systems requires communication protocols. The protocols must provide common interfaces to all computers in the system. Many distributed systems require communication of state information to maintain efficient operation. **State information** consists of data that describes the status of one or more resources. Improper state information could lead to inconsistency; transmitting state information too often can flood the network and reduce scalability Distributed system designers must determine what state information to maintain and how often to make such information known to the system.[1]

Distributed systems can be susceptible to attacks by malicious users if they rely on insecure communications media (i.e., a public network). To improve security, a distributed system should allow only authorized users to access resources and ensure that information transmitted over the network is readable only by the intended recipients. Also, because many users or objects may request resources, the system must provide mechanisms to protect resources from attack. Security and protection are discussed in Chapter 19.

Self Review

1. What facts should distributed system designers consider when designing state information?
2. Give examples of poor state-information design.

Ans: **1)** Distributed system designers need to decide whether the system should keep state information, what state information to maintain and how often to make such information known to the system. **2)** Poor state-information design may keep multiple entries for the status of the same resource, leading to inconsistency; it may cause state information to be transmitted too often, which could flood the network and restrict scalability.

17.2.3 Reliability and Fault Tolerance

The failure of one or more resources on a single machine may cause the whole system to fail or may affect the performance of processes running on the system. A distributed system is more likely to suffer failures than a single-machine system because it has more components that might malfunction.[2]

Distributed systems implement fault tolerance by providing replication of resources across the system. The failure of any one computer will not affect the availability of the system's resources. For example, a distributed file system might keep copies of the same file at different servers. If a user is using a file on a server that crashes, then the distributed file system can direct future requests to a server with a copy of the original file. Replication offers users increased reliability and availability over single-machine implementations.

This comes at a cost. Distributed system designers must develop software that detects and reacts to system failures. Furthermore, designers must provide mechanisms to ensure consistency among the state information at different machines. Such systems must also be equipped to reintegrate failed resources, once they have been repaired.

Self Review

1. Why is a distributed system more likely to suffer faults than a single machine?
2. What is the side effect of increasing reliability and fault tolerance?

Ans: **1)** Distributed systems simply have more components that might malfunction.
2) Increasing reliability and fault tolerance makes designing distributed systems more complex. The designer must provide mechanisms to switch from a failed machine to a working machine and keep the copies of the duplicated resources consistent across the system.

17.2.4 Transparency

One goal of a distributed system is to provide **transparency** by hiding the distribution aspects from users of the system. Access to a file system that is distributed across several remote computers should be no different than access to a local file system. The user of a distributed file system should be able to access its resources without knowing about the communication between the processes, which process runs the user request and the physical location of the resources.

The ISO Open Distributed Processing Reference Model defines eight types of transparency a distributed system can provide:[3]

- access transparency
- location transparency
- failure transparency
- replication transparency
- persistence transparency
- migration transparency
- relocation transparency
- transaction transparency

Access transparency hides the details of networking protocols that enable communication between distributed computers. It also provides a universal means to access data stored in disparate data formats throughout a system.

Location transparency builds on access transparency to hide the location of resources in the distributed system from those attempting to access them. A distributed file system that provides location transparency allows access to remote files as if they were local files.

Failure transparency is the method by which a distributed system provides fault tolerance. If one or more resources or computers in the network fail, users of the system will be aware only of the reduced performance. Failure transparency is typically implemented by **replication** or checkpoint/recovery. Under replication, a system provides multiple resources that perform the same function. Even if all but one of a set of replicated resources fails, a distributed system can continue to function. A system that employs checkpointing periodically stores the state of an object (such as a process) such that it can be restored (i.e., recovered) if a failure in the distributed system results in the loss of the object.

Replication transparency hides the fact that multiple copies of a resource are available in the system; all access to a group of replicated resources occurs as if there were one such resource available. **Persistence transparency** hides the information about where the resource is stored—memory or disk.

Migration and **relocation transparency** both hide the movement of components of a distributed system. Migration transparency masks the movement of an object from one location to another in the system, such as the movement of a file from one server to another. Relocation transparency masks the relocation of an